PROJECT OVERVIEW:

- JUST SKAN aims to develop a real time application that helps the user(staff) to evaluate OMR sheets for MCQ type exams .
- A login module through which user can enter into application if they have registered id . User can create a new id by clicking sign up button which opens sign up module.
- Same user can create multiple ids for students of different grades /classes/groups in order to store students of different groups in different tables.
- Add student module , which allows user to add new students to the students table of that particular user id through which the user has logged in.
- Update student module which allows user to change/update student details.
- Remove student module which allows user to delete student from the students table .
- Create test module , which allows user to create new test by specifying test name , number of questions , test date , and answer key which can be entered through a sub module after clicking a button. This test details will be stored in a table of the particular id through which the user has logged in.
- Edit test module which allows user to alter any corrections in test date / number of questions / answer key .
- Upload OMR module which opens a small window in which the user has to specify the test name for which OMR is going to be uploaded and the dimensions of OMR sheet (number of rows and columns). After filling these details the list of student ids ,names in the table will be displayed along with a label to show whether their omr sheet is uploaded or not and a button to upload or re upload it.
- On clicking the button , user can select the image of OMR of that student from files and then click the corner points of OMR sheet in the corresponding order specified in a sample image .After that user can adjust a thresh hold value to improve accuracy. This value which may vary for each image depending upon its environment and brightness. User can click proceed , which generates a small report on test marks , correct , incorrect , unattempted questions . User can click save to store the details in students table of corresponding id through which they logged in.

- View tests module which allows the user to view test details of all tests created in the corresponding id through which they logged in . They can click a button to view the answer key of the tests.
- View student marks module which opens two modules one by one in which user can select only the students he/she wish to see marks and select only the tests which he/she needs to analyse. Then in a module the tests appear in columns and student id , name appear in rows . The marks of student appear in each cell. And a final column is created which shows the average percentage marks of each student for the tests selected. User can sort the order of table by name or marks .Both ascending and descending order is available.
- Edit student marks module which opens a module in which user has to specify the test name , and the specific question number for which mark has to be altered and select students for whom mark has to be altered and on clicking a button a small window opens in which user has to specify what should be the new state of the question (attempted correct /attempted wrong /unattempted). According to the new state entered marks will be changed for all selected students. The change in marks for a particular student depends on both his /her old state of this question and new state of the question.
- Create report module which opens a window to ask the student id and test name then shows a report of the student in the particular test.

SYSTEM STUDY:

1.PROBLEM DESCRIPTION:

- The problem the application tries to solve is the difficulty of correcting OMR sheets for MCQ type tests like JEE, NEET ,etc manually ,which consumes a lot of time and some times human error may occur in this process.
- This project simplifies the work of OMR sheet evaluators and also stores the marks secured by students in database.

2.EXISTING SYSTEM:

- There are OMR sheet scanners available in the market ,which are quite costly .
- So most schools and entrance exam coaching institutes appoint more staffs to correct it , which is followed in our school.

3.PROPOSED SYSTEM:

- The proposed system is a real time application which allows the user(staff) to store student details and marks in a table and test details in another table.
- User can create multiple login id s to create multiple tables for students of various groups / streams.
- User can enter the answer key for a test and then upload OMR sheet photos for all students ,after selecting the corners of sheet and adjusting few parameters, the OMR sheet will be auto evaluated and the marks will be stored in table, which can be viewed whenever.

4.FUTURE PROPOSAL:

- The future proposal of JUST SKAN is to reduce the steps between uploading photo and auto evaluation and making it more easy to use.
- Adding additional student and test details to be stored in database.
- Developing this application in android platform as an app.

1) IMPORTS AND SOME FUNCTIONS

```
#imports
import time
import tkinter as tk
from tkinter import *
from tkinter import filedialog
from tkinter import ttk
import numpy as np
import cv2
from tkinter import messagebox
from tkcalendar import Calendar
from datetime import datetime
import mysql.connector  as ms
from PIL import ImageTk,Image
# background images
img=Image.open("appbgimg11.jpg")
img=img.resize((520,520),Image.LANCZOS)
img=ImageTk.PhotoImage(img)
img2=Image.open("appbgimg12.jpg")
img2=img2.resize((1200,800),Image.LANCZOS)
img2=ImageTk.PhotoImage(img2)
img10=Image.open("appbgimg2.png")
img10=img10.resize((470,740),Image.LANCZOS)
img10=ImageTk.PhotoImage(img10)
img3=Image.open("appbgimg7.jpg")
img3=img3.resize((600,600),Image.LANCZOS)
img3_=img3.resize((600,640),Image.LANCZOS)
img3_=ImageTk.PhotoImage(img3_)
img3=ImageTk.PhotoImage(img3)
img4=Image.open("appbgimg11.jpg")
img4=img4.resize((600,600),Image.LANCZOS)
img4=ImageTk.PhotoImage(img4)
omr90=Image.open("omr90NEW.png")
omr90=omr90.resize((500,500),Image.LANCZOS)
omr90=ImageTk.PhotoImage(omr90)
omr45=Image.open("omr45NEW.png")
omr45=omr45.resize((500,500),Image.LANCZOS)
omr45=ImageTk.PhotoImage(omr45)
omr180=Image.open("omr180NEW.png")
omr180=omr180.resize((500,500),Image.LANCZOS)
omr180=ImageTk.PhotoImage(omr180)
omr30=Image.open("omr30NEW.png")
omr30=omr30.resize((500,500),Image.LANCZOS)
omr30=ImageTk.PhotoImage(omr30)
```

```
omricon=Image.open("omricon.jpg")
omricon=omricon.resize((200,200),Image.LANCZOS)
omricon=ImageTk.PhotoImage(omricon)

def breakmultilines(s,w=30):
    news=""
     c=0
    for i in s:
        news+=i
        c+=1
        if c%w==0:
            news+="\n"
     return news
def dict_to_str(d):
    l=[]
    for k in d:
        l.append(k)
    l2=sorted(l)
    anskey_str=""
     for k in l2:
        anskey_str+=" "+str(k)+d[k]
    return anskey_str
def str_to_dict(s):
    d={}
    j=0

    l=s.split()
    qno=1
    for i in l:
        d[qno]=i[-1]
        qno+=1

    return d
#variables
r=IntVar()
signup_window = None
main_window=None
add_stu_window=None
upd_stu_window=None
rem_stu_window=None
new_stu_name_add=None
sid_add=None
new_stu_name_upd=None
new_test_window=None
enter_anskey_window=None
edit_ans_key_window=None
ask_test_name_window=None
```

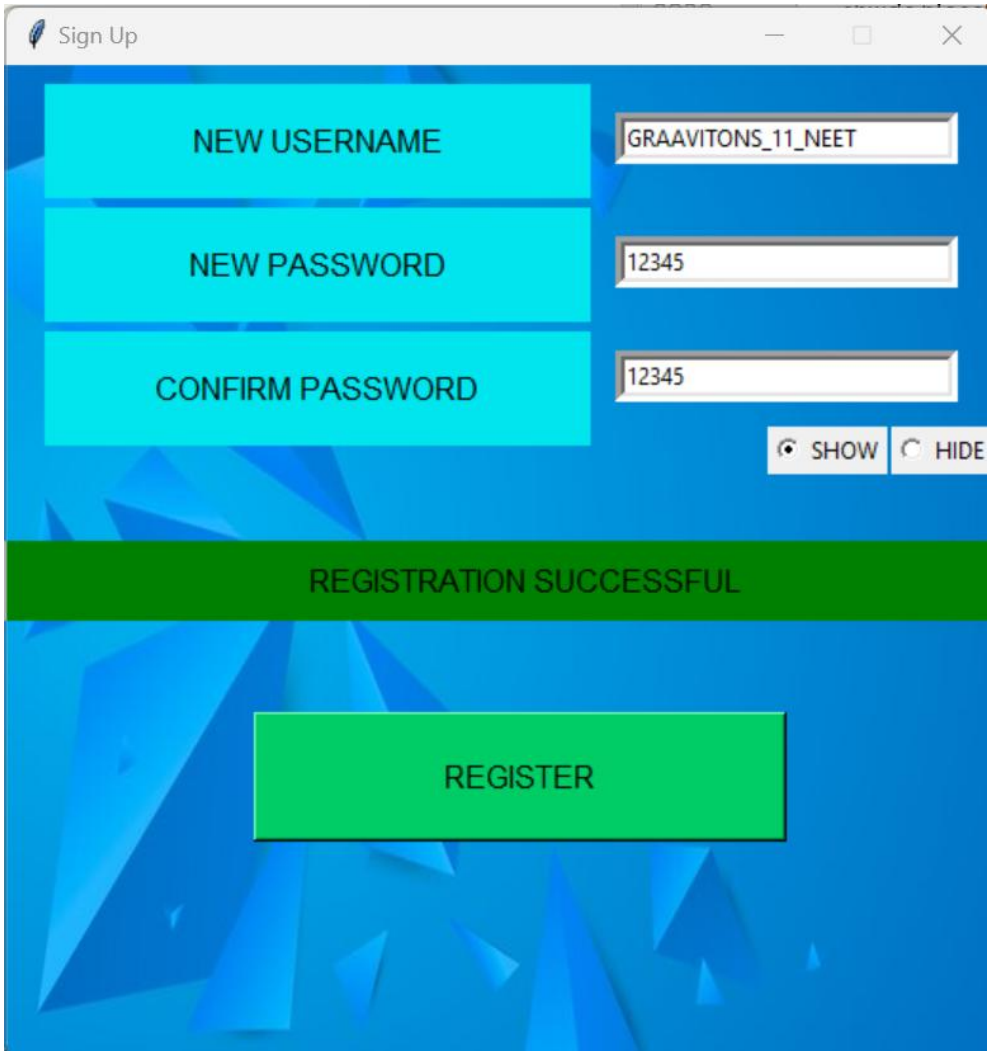
```
view_tests_window=None
viewanskeywindow=None
edit_test_window=None
ask_new_state_of_q_window=None
upload_omr_photo_window= None
omr_crop_window=None
sid_upd=None
sid_rem=None
generate_report_1window=None
generate_report_2window=None
edit_marks_window=None
imgtresh=None
omr_type=None
img_omr=None
var=StringVar()
stu_report_window=None
select_stu_window=None
select_test_window=None
view_stu_markwindow=None
test_name_toupdatemarks=None
sort=StringVar()
sort.set("NAMEASC")
threshhold_value = 100
stu_shadedoptions=[]
selected_students=[]
selected_tests=[]
```

2)SIGN UP MODULE

```
def register_user():
    global nune
    global npwde
    global cpwde
    new_username = nune.get().replace(" ","_")
    new_password = npwde.get().replace(" ","_")
    confirm_password=cpwde.get().replace(" ","_")
    db = ms.connect(
        host="localhost",
        user="root",
        password="arunesh7",
        database="CS_PROJECT"    )
    cursor= db.cursor()
    if new_password==confirm_password:
        cursor.execute("SELECT * FROM USERS WHERE U_NAME = %s",(new_username,))
        user = cursor.fetchone()
        if user:
            register_status_label.config(text="USERNAME ALREADY EXISTS",bg="red")
            nune.delete(0, END)
        elif  not new_username or not new_password :
            register_status_label.config(text="PLEASE ENTER ALL DETAILS ",bg="red")
        else:
            insert_query = "INSERT INTO USERS VALUES (%s, %s);"
            data = (new_username, new_password)
            cursor.execute(insert_query, data)
            create_stu_table="CREATE TABLE "+new_username+"STUDENTS"+"(STU_ID
VARCHAR(15),STU_NAME VARCHAR(25));"
            create_anskey_table="CREATE TABLE "+new_username+"ANS_KEY (TEST_NAME
VARCHAR(30),ANS_KEY VARCHAR(1024),TEST_DATE DATE)"

            cursor.execute(create_stu_table)
            cursor.execute(create_anskey_table)

            db.commit()
            register_status_label.config(text="REGISTRATION SUCCESSFUL",bg="green")
    else:
        register_status_label.config(text="CHECK YOUR PASSWORD",bg="red")
    db.close()


def signup_window():
    global signup_window
    signup_window = tk.Toplevel(window,)
    signup_window.title("Sign Up")
```

```
    signup_window.geometry("520x520")

    signup_window.resizable(height=False,width=False)
    bgl2 = tk.Label(signup_window, image=img)
    bgl2.place(x=0, y=0, relwidth=1, relheight=1)
    global nune, npwde, register_status_label,cpwde
    nunl = tk.Label(signup_window, text="NEW
USERNAME",width=27,height=2,pady=10,padx=20,bg="turquoise2",font=("Brittanic
Bold",12),).place(x=20,y=10)
    nune = tk.Entry(signup_window,width=28,borderwidth=5)
    nune.place(x=320,y=25)
    npwdl = tk.Label(signup_window, text="NEW
PASSWORD",width=27,height=2,pady=10,padx=20,bg="turquoise2",font=("Brittanic
Bold",12),).place(x=20,y=75)
    npwde = tk.Entry(signup_window, show="*",width=28,borderwidth=5)
    npwde.place(x=320,y=90)
    cpwdl = tk.Label(signup_window, text="CONFIRM
PASSWORD",width=27,height=2,pady=10,padx=20,bg="turquoise2",font=("Brittanic
Bold",12),).place(x=20,y=140)
    cpwde = tk.Entry(signup_window, show="*",width=28,borderwidth=5)
    cpwde.place(x=320,y=150)
    register_button = tk.Button(signup_window, text="REGISTER",
command=register_user,width=30,height=3,bg="springgreen3",font=("Brittanic Bold",12))
    register_button.place(x=130,y=340)
    register_status_label = tk.Label(signup_window, text="ENTER YOUR
DETAILS",width=60,height=2,font=("Brittanic Bold",12))
    register_status_label.place(x=0,y=250)
    show_pwd1 = tk.Radiobutton(signup_window, variable=r, value=2, text="SHOW",
command=lambda: showpwd(r.get()), )
    show_pwd1.place(x=400,y=190)
    hide_pwd1 = Radiobutton(signup_window, variable=r, value=3, text="HIDE", command=lambda:
showpwd(r.get()), )
    hide_pwd1.place(x=465,y=190)

def showpwd(a):
    global pwde,npwde,cpwde
    if a==1:
        pwde.configure(show="")
    elif a==0:
        pwde.configure(show="*")
    elif a==2:
        npwde.configure(show="")
        cpwde.configure(show="")
    else:
        npwde.configure(show="*")
        cpwde.configure(show="*")
```

```
mysql> select * from users;
+--------------------+-------+
| U_NAME             | PWD   |
+--------------------+-------+
| GRAAVITONS_11_JEE  | 12345 |
| GRAAVITONS_12_JEE  | 12345 |
| GRAAVITONS_12_NEET | 12345 |
+--------------------+-------+
3 rows in set (0.00 sec)
```

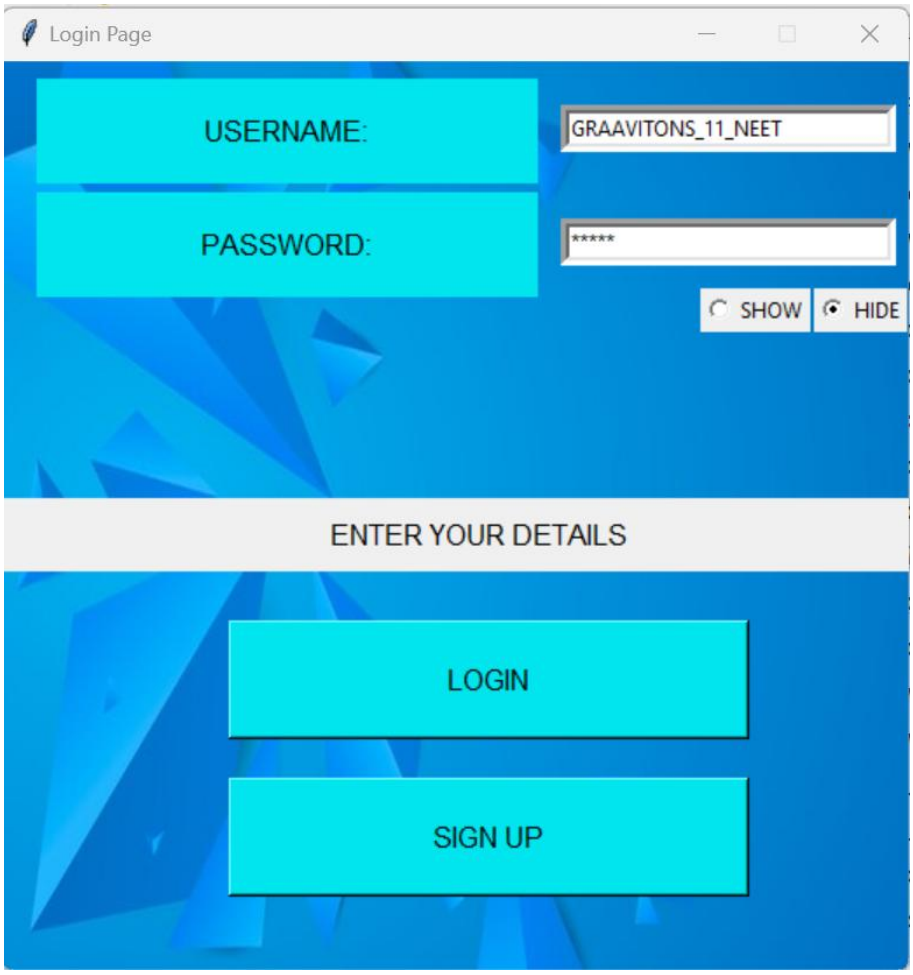


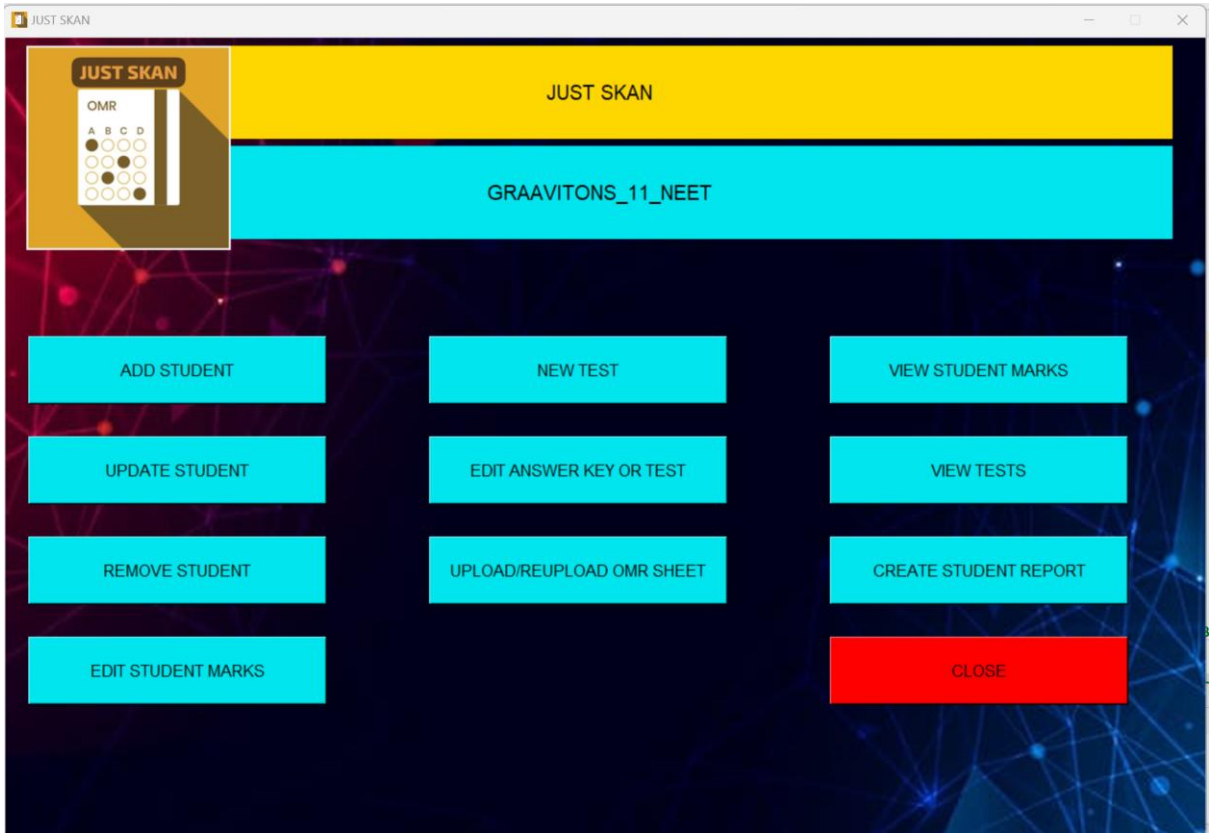
```
mysql> select * from users;
+--------------------+-------+
| U_NAME             | PWD   |
+--------------------+-------+
| GRAAVITONS_11_JEE  | 12345 |
| GRAAVITONS_11_NEET | 12345 |
| GRAAVITONS_12_JEE  | 12345 |
| GRAAVITONS_12_NEET | 12345 |
+--------------------+-------+
4 rows in set (0.00 sec)
```

```
mysql> SHOW TABLES;
+---------------------------+
| Tables_in_cs_project      |
+---------------------------+
| graavitons_11_jeeans_key  |
| graavitons_11_jeestudents |
| graavitons_11_neetans_key |
| graavitons_11_neetstudents |
| graavitons_12_jeeans_key  |
| graavitons_12_jeestudents |
| graavitons_12_neetans_key |
| graavitons_12_neetstudents |
| users                     |
+---------------------------+
9 rows in set (0.00 sec)
```

3)LOGIN MODULE

```
window = tk.Tk()
window.title("Login Page")
window.geometry("520x520")
window.resizable(height=False,width=False)

bgl=tk.Label(window,image=img)
bgl.place(x=0,y=0,relwidth=1,relheight=1)
#LABELS AND ENTRIES
unl = tk.Label(window,
text="USERNAME:",width=27,height=2,pady=10,padx=20,bg="turquoise2",font=("Brittanic
Bold",12),).place(x=20,y=10)
une = tk.Entry(window,width=30,borderwidth=5,)
une.place(x=320,y=25)
pwdl= tk.Label(window,
text="PASSWORD:",width=27,height=2,pady=10,padx=20,bg="turquoise2",font=("Brittanic
Bold",12)).place(x=20,y=75)
pwde= tk.Entry(window, show="*",width=30,borderwidth=5)
pwde.place(x=320,y=90)
login_status_label = tk.Label(window, text="ENTER YOUR
DETAILS",width=60,height=2,font=("Brittanic Bold",12))
login_status_label.place(x=0,y=250)
#LOGIN PAGE BUTTONS
show_pwd=tk.Radiobutton(window,variable=r,value=1,text="SHOW",command=lambda
:showpwd(r.get()),)
show_pwd.place(x=400,y=130)
hide_pwd=Radiobutton(window,variable=r,value=0,text="HIDE",command=lambda
:showpwd(r.get()),)
hide_pwd.place(x=465,y=130)
signup_button = tk.Button(window, text="SIGN UP",
command=signup_window,width=32,height=3,bg="turquoise2",font=("Brittanic Bold",12))
signup_button.place(x=130,y=410)
login_button = tk.Button(window, text="LOGIN",
command=check_login,width=32,height=3,bg="turquoise2",font=("Brittanic Bold",12))
login_button.place(x=130,y=320)

window.mainloop()
```


Login Page
USERNAME:
GRAAVITONS_11_NEET
PASSWORD:
*****
SHOW
HIDE
ENTER YOUR DETAILS
LOGIN
SIGN UP

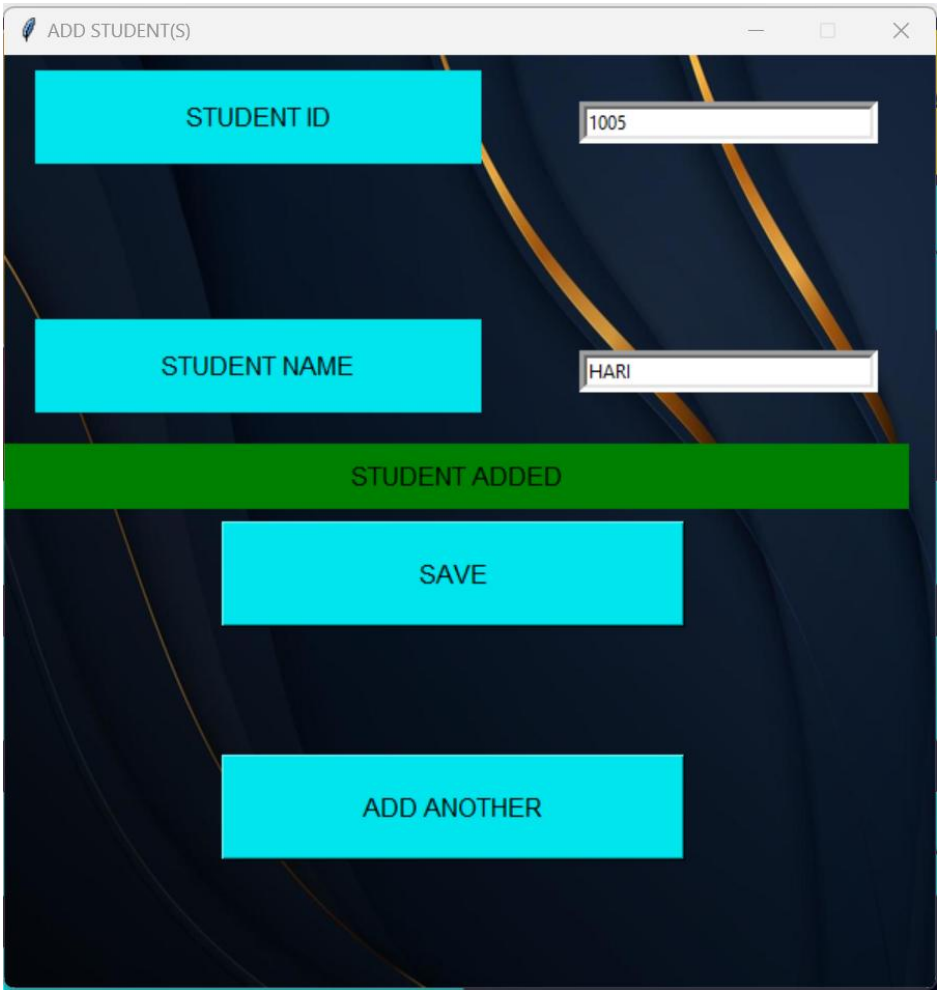

JUST SKAN
JUST SKAN
OMR
A B C D
JUST SKAN
GRAAVITONS_11_NEET
ADD STUDENT
NEW TEST
VIEW STUDENT MARKS
UPDATE STUDENT
EDIT ANSWER KEY OR TEST
VIEW TESTS
REMOVE STUDENT
UPLOAD/REUPLOAD OMR SHEET
CREATE STUDENT REPORT
EDIT STUDENT MARKS
CLOSE

4)ADD STUDENT MODULE

```
def Add_stu():


    global new_stu_name_add, sid_add, une,sid_entry,sn_entry,messagelabel

    def save_new_stu():
        sid_add = sid_entry.get().replace(" ","_")
        new_stu_name_add = sn_entry.get().replace(" ","_")

        un=str(une.get().replace(" ","_"))+"students"
            stu_name = new_stu_name_add

        db = ms.connect(
            host="localhost",
            user="root",
            password="arunesh7",
            database="CS_PROJECT"
            )
        cursor = db.cursor()

        if stu_name and sid_add:
            cursor.execute("SELECT * FROM "+un+" WHERE stu_id ="+sid_add+";")
            a=cursor.fetchone()
            if a:
                messagelabel.configure(bg="red",text="STUDENT ID ALREADY EXISTS")
                sid_entry.delete(0,END)
            else:
                cursor.execute(f"insert into {un}(STU_ID,STU_NAME)
values('{sid_add}','{stu_name}');")
                messagelabel.configure(bg="green", text="STUDENT ADDED")
                add_another_button.configure(state="normal")
            db.commit()
        else:
            messagelabel.configure(bg="red",text="ENTER ALL DETAILS")
    def add_another_stu():
        sid_entry.delete(0, END)
        sn_entry.delete(0, END)
        add_another_button.configure(state="disabled")
        messagelabel.configure(bg="white",text="ENTER YOUR DETAILS")


    global add_stu_window,img3
    add_stu_window=Toplevel(window,)
    add_stu_window.title("ADD STUDENT(S)")
```

```
    add_stu_window.geometry("600x600")
    add_stu_window.resizable(height=False,width=False)
    bgl3 = tk.Label(add_stu_window, image=img3)
    bgl3.place(x=0, y=0, relwidth=1, relheight=1)

    sidl=Label(add_stu_window,text="STUDENT
ID",width=27,height=2,pady=10,padx=20,bg="turquoise2",font=("Brittanic
Bold",12),).place(x=20,y=10)
    sid_entry=Entry(add_stu_window,width=30,borderwidth=5,)
    sid_entry.place(x=370,y=30)
    snl = Label(add_stu_window, text="STUDENT NAME", width=27, height=2, pady=10,
padx=20, bg="turquoise2",font=("Brittanic Bold", 12), ).place(x=20, y=170)
    sn_entry = Entry(add_stu_window, width=30, borderwidth=5, )
    sn_entry.place(x=370, y=190)
    save_stu_button=Button(add_stu_window,text="SAVE",
command=save_new_stu,width=32,height=3,bg="turquoise2",font=("Brittanic Bold",12))
    save_stu_button.place(x=140,y=300)
    add_another_button=Button(add_stu_window,text="ADD ANOTHER",
command=add_another_stu,width=32,height=3,bg="turquoise2",font=("Brittanic
Bold",12),state="disabled")
    add_another_button.place(x=140,y=450)
    messagelabel=Label(add_stu_window,text="ENTER
DETAILS",width=64,height=2,font=("Brittanic Bold",12))
    messagelabel.place(x=0,y=250)
```

```
mysql> SELECT * FROM GRAAVITONS_11_NEETSTUDENTS;
+--------+----------+
| STU_ID | STU_NAME |
+--------+----------+
| 1001   | ASHWIN   |
| 1002   | AKASH    |
| 1003   | BHARAT   |
| 1004   | KAPINESH |
+--------+----------+
4 rows in set (0.00 sec)
```

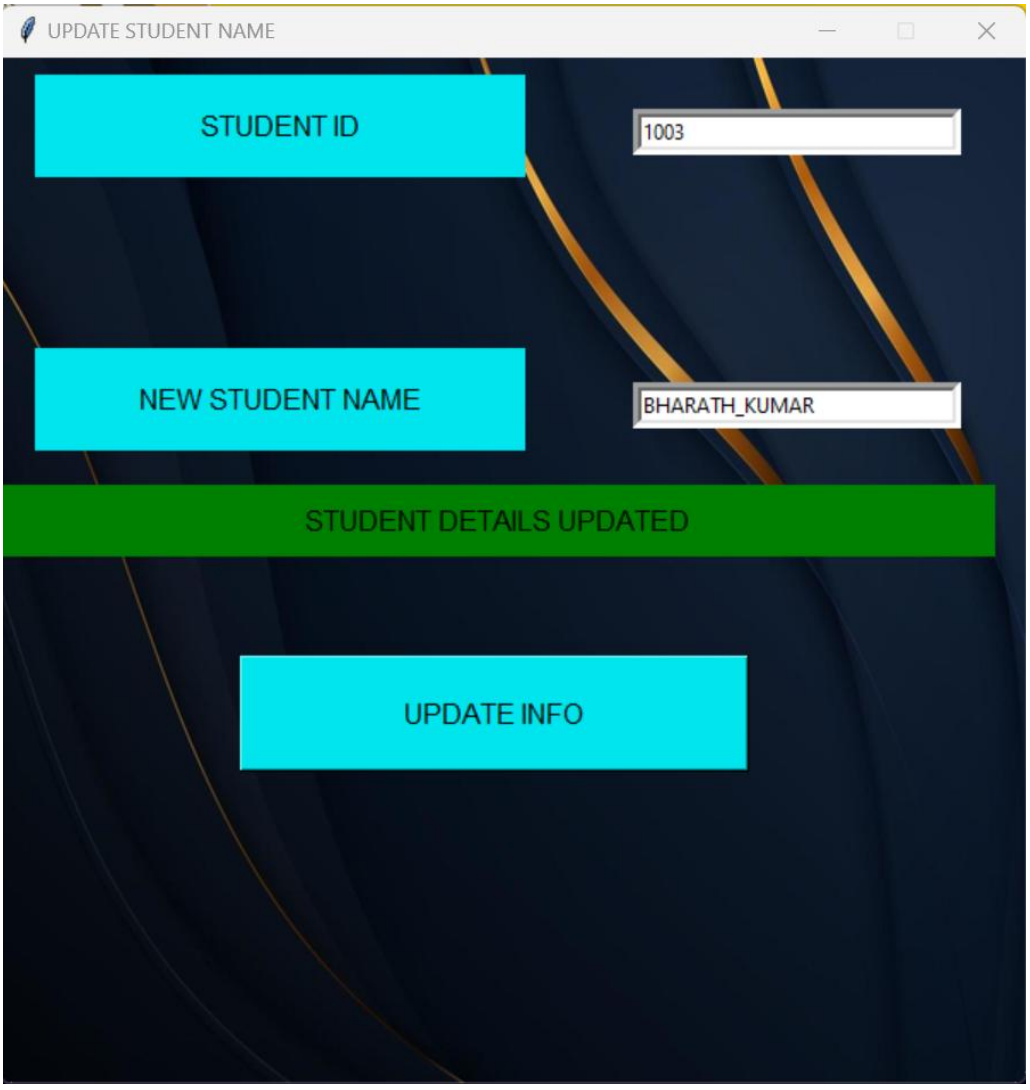



```
mysql> SELECT * FROM GRAAVITONS_11_NEETSTUDENTS;
+--------+----------+
| STU_ID | STU_NAME |
+--------+----------+
| 1001   | ASHWIN   |
| 1002   | AKASH    |
| 1003   | BHARAT   |
| 1004   | KAPINESH |
| 1005   | HARI     |
+--------+----------+
5 rows in set (0.00 sec)
```

5)UPDATE STUDENT MODULE

```
def Update_stu():

    global upd_stu_window, img3,new_stu_name_upd, sid_upd,une
    def update_old_stu():
        sid_upd = sid_entry.get().replace(" ","_")
        new_stu_name_upd = sn_entry.get().replace(" ","_")
        un = str(une.get().replace(" ","_")) + "students"


            db = ms.connect(
            host="localhost",
            user="root",
            password="arunesh7",
            database="CS_PROJECT"
            )
        cursor = db.cursor()

        if sid_upd and new_stu_name_upd:
            cursor.execute("SELECT * FROM " + un + " WHERE stu_id =" + str(sid_upd )+ ";")
            a = cursor.fetchone()
            if a:
                cursor.execute("update "+ un +f" set stu_name = '{new_stu_name_upd}' where
stu_id = '{sid_upd}';")
                messagelabel2.configure(bg="green",text="STUDENT DETAILS UPDATED")
                db.commit()
            else:
                messagelabel2.configure(bg="red",text="STUDENT ID DOES NOT EXIST")
                sid_entry.delete(0,END)
        else:
            messagelabel2.configure(bg="red", text="ENTER ALL DETAILS")
    upd_stu_window = Toplevel(window, )
    upd_stu_window.title("UPDATE STUDENT NAME")
    upd_stu_window.geometry("600x600")
    upd_stu_window.resizable(height=False, width=False)
    bgl4 = tk.Label(upd_stu_window, image=img3)
    bgl4.place(x=0, y=0, relwidth=1, relheight=1)

    sidl = Label(upd_stu_window, text="STUDENT ID", width=27, height=2, pady=10,
padx=20, bg="turquoise2",
                 font=("Brittanic Bold", 12), ).place(x=20, y=10)
    sid_entry = Entry(upd_stu_window, width=30, borderwidth=5, )
    sid_entry.place(x=370, y=30)
    nsnl = Label(upd_stu_window, text="NEW STUDENT NAME", width=27, height=2,
pady=10, padx=20, bg="turquoise2",font=("Brittanic Bold", 12), ).place(x=20, y=170)
    sn_entry = Entry(upd_stu_window, width=30, borderwidth=5, )
```

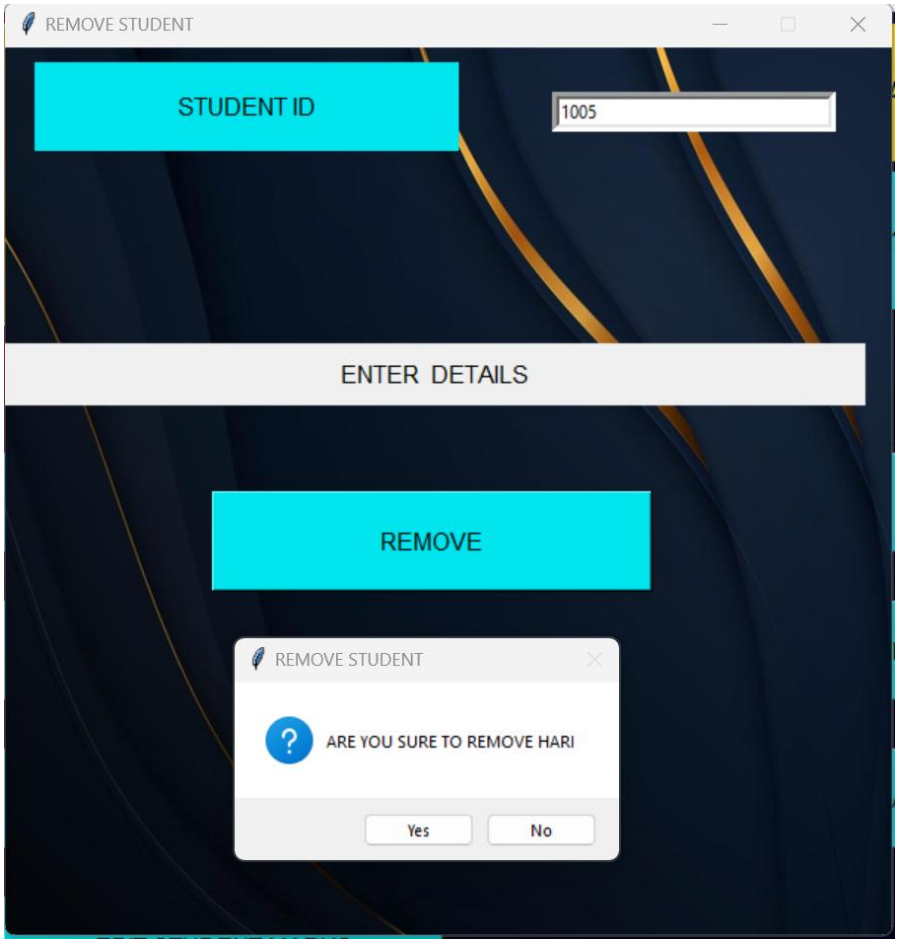
```
    sn_entry.place(x=370, y=190)
    upd_stu_button = Button(upd_stu_window, text="UPDATE INFO",
command=update_old_stu, width=32, height=3, bg="turquoise2", font=("Brittanic Bold",
12))
    upd_stu_button.place(x=140, y=350)
    messagelabel2 = Label(upd_stu_window, text="ENTER  DETAILS", width=64, height=2,
font=("Brittanic Bold", 12))
    messagelabel2.place(x=0, y=250)
```

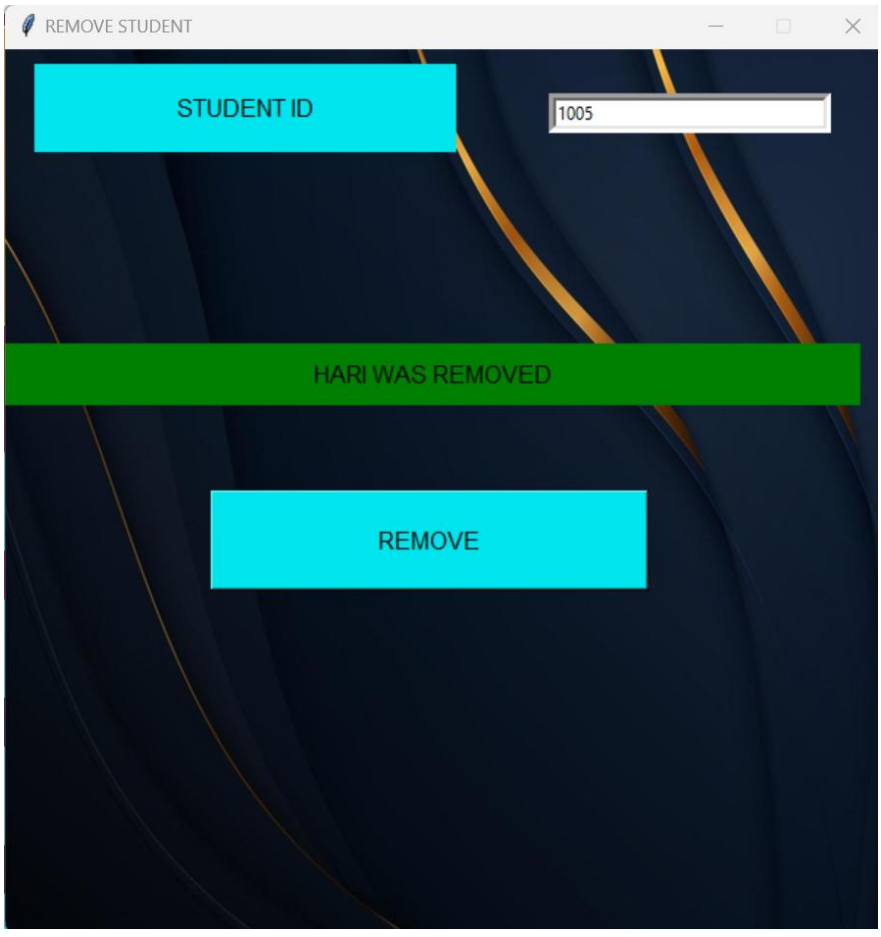



```
mysql> SELECT * FROM GRAAVITONS_11_NEETSTUDENTS;
+--------+---------------+
| STU_ID | STU_NAME      |
+--------+---------------+
| 1001   | ASHWIN        |
| 1002   | AKASH         |
| 1003   | BHARATH_KUMAR |
| 1004   | KAPINESH      |
| 1005   | HARI          |
+--------+---------------+
```

6) REMOVE STUDENT MODULE

```
def Remove_stu():
    global rem_stu_window, img3,sid_rem,une
    def remove_student():
        sid_rem = sid_entry.get().replace(" ","_")
        un = str(une.get().replace(" ","_")) + "students"

            db = ms.connect(
            host="localhost",
            user="root",
            password="arunesh7",
            database="CS_PROJECT")

        cursor = db.cursor()
        if sid_rem:
            cursor.execute("SELECT * FROM " + un + " WHERE stu_id =" + str(sid_rem) +
";")
            a = cursor.fetchone()
            if a:
                ans=messagebox.askyesno("REMOVE STUDENT","ARE YOU SURE TO REMOVE
"+str(a[1]),)
                if ans:
                    messagelabel3.configure(bg="green", text=str(a[1]) + " WAS REMOVED")
                    cursor.execute("delete from "+un+ f" where stu_id={sid_rem} ;")
                    db.commit()
            else:
                messagelabel3.configure(bg="red",text="STUDENT ID DOES NOT EXIST")
    rem_stu_window = Toplevel(window, )
    rem_stu_window.title("REMOVE STUDENT")
    rem_stu_window.geometry("600x600")
    rem_stu_window.resizable(height=False, width=False)
    bgl5 = tk.Label(rem_stu_window, image=img3)
    bgl5.place(x=0, y=0, relwidth=1, relheight=1)
    sidl = Label(rem_stu_window, text="STUDENT ID", width=27, height=2, pady=10,
padx=20, bg="turquoise2",font=("Brittanic Bold", 12), ).place(x=20, y=10)
    sid_entry = Entry(rem_stu_window, width=30, borderwidth=5, )
    sid_entry.place(x=370, y=30)
    rem_stu_button = Button(rem_stu_window, text="REMOVE", command=remove_student,
width=32, height=3, bg="turquoise2",font=("Brittanic Bold", 12))
    rem_stu_button.place(x=140, y=300)
    messagelabel3 = Label(rem_stu_window, text="ENTER  DETAILS", width=64, height=2,
font=("Brittanic Bold", 12))
    messagelabel3.place(x=0, y=200)
```


REMOVE STUDENT
STUDENT ID
1005
ENTER DETAILS
REMOVE
REMOVE STUDENT
ARE YOU SURE TO REMOVE HARI
Yes
No


REMOVE STUDENT
STUDENT ID
1005
HARI WAS REMOVED
REMOVE

```
mysql> SELECT * FROM GRAAVITONS_11_NEETSTUDENTS;
+--------+---------------+
| STU_ID | STU_NAME      |
+--------+---------------+
| 1001   | ASHWIN        |
| 1002   | AKASH         |
| 1003   | BHARATH_KUMAR |
| 1004   | KAPINESH      |
+--------+---------------+
4 rows in set (0.00 sec)
```

7) CREATE NEW TEST MODULE

```
def new_test():

   def enter_anskey():
      global enter_anskey_window, img3,no_of_q
      try:
         no_of_q = int(nqe.get())

         if no_of_q > 0:
            status_label.configure(bg="green", text="ENTER ANS KEY,SAVE AND CREATE
TEST")

            enter_anskey_window = Toplevel(window, )

            enter_anskey_window.geometry("600x600")
            enter_anskey_window.title("ENTER ANSWER KEY")
            enter_anskey_window.resizable(height=False, width=False)
            bgl6 = tk.Label(enter_anskey_window, image=img3)
            bgl6.place(x=0, y=0, relwidth=1, relheight=1)
            global q
            q = 1

            def next():
               global q

               if q < no_of_q:
                  q += 1
                  qno_label.configure(text="QNO" + str(q))

            def previous():
               global q
               if q > 1:
                  q -= 1
                  qno_label.configure(text="QNO" + str(q))

            def select_qno(e):
               global q
               r = lb1.get(ACTIVE).split()[0]
               q = int(r[3:])

               qno_label.configure(text="QNO" + str(q))

            qno_label = Label(enter_anskey_window, text="QNO" + str(q), width=27,
height=2, pady=10, padx=20,
                              bg="turquoise2",
                              font=("Brittanic Bold", 12), )
```

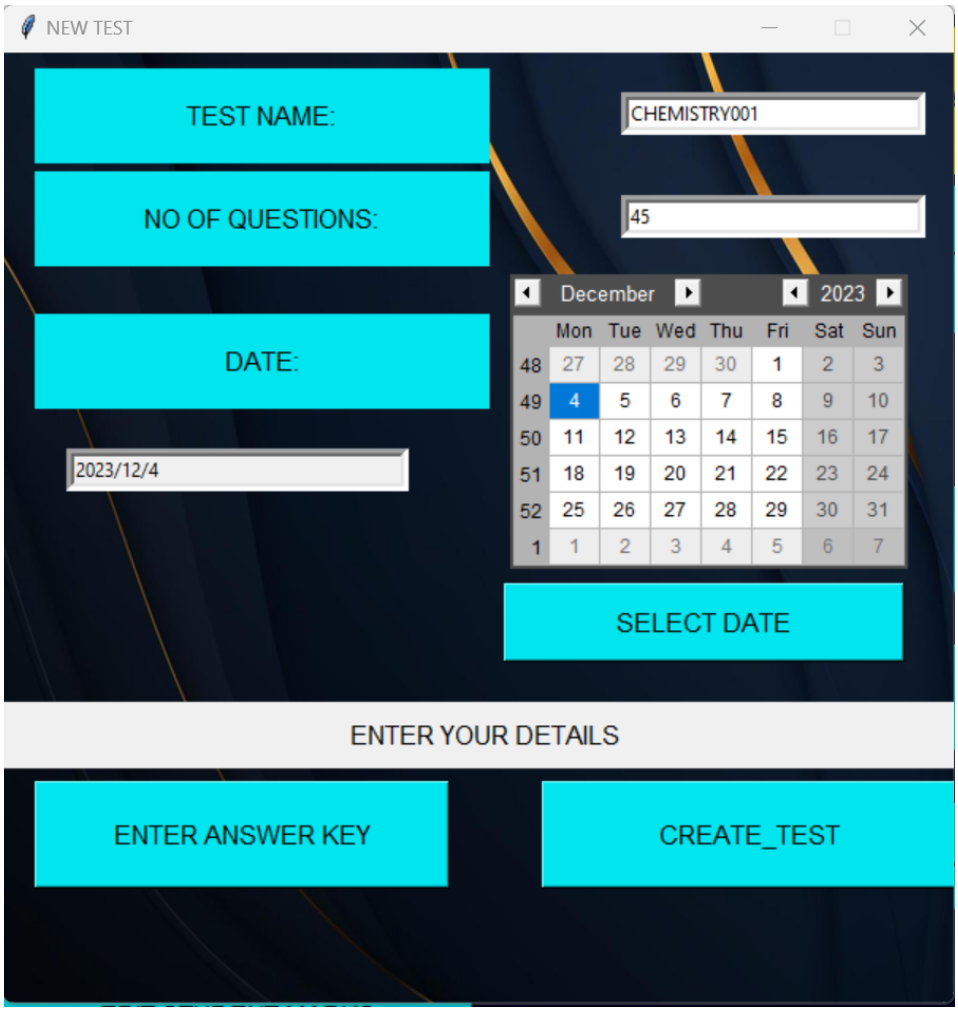
```
            qno_label.place(x=165, y=10)
            next_button = Button(enter_anskey_window, text="NEXT", command=next,
width=12, height=2,
                                 bg="turquoise2",
                                 font=("Brittanic Bold", 12), )
            next_button.place(x=470, y=10)
            previous_button = Button(enter_anskey_window, text="PREVIOUS",
command=previous, width=12, height=2,
                                   bg="turquoise2",
                                   font=("Brittanic Bold", 12), )
            previous_button.place(x=30, y=10)
            frame = Frame(enter_anskey_window, bg="snow3", width=150, height=400)
            frame.place(x=0, y=100)

            lb1 = Listbox(frame, width=20, height=24, bg="cyan", )
            lb1.place(x=0, y=0)

            def store_anskey():
               global anskey, r2,anskey_str
               optn=r2.get()
               anskey[q] = optn

               anskey_str=dict_to_str(anskey)
               val=lb1.get(q-1).split()[0]
               val+=" OPTION "+optn
               lb1.delete(q-1)
               lb1.insert(q-1,val)



            A_button = Radiobutton(enter_anskey_window, variable=r2, value="A",
text="OPTION A",
activebackground="springgreen",bg="snow3",width=10,height=2,font=("Brittanic Bold",18),)
            A_button.place(x=240, y=200)
            B_button = Radiobutton(enter_anskey_window, variable=r2, value="B",
text="OPTION B", activebackground="springgreen"
,bg="snow3",width=10,height=2,font=("Brittanic Bold",18))
            B_button.place(x=240, y=250)
            C_button = Radiobutton(enter_anskey_window, variable=r2, value="C",
text="OPTION C",
activebackground="springgreen",bg="snow3",width=10,height=2,font=("Brittanic Bold",18))
            C_button.place(x=240, y=300)
            D_button = Radiobutton(enter_anskey_window, variable=r2, value="D",
text="OPTION D",
activebackground="springgreen",bg="snow3",width=10,height=2,font=("Brittanic Bold",18))
            D_button.place(x=240, y=350)
            N_button = Radiobutton(enter_anskey_window, variable=r2, value="N", text="
```

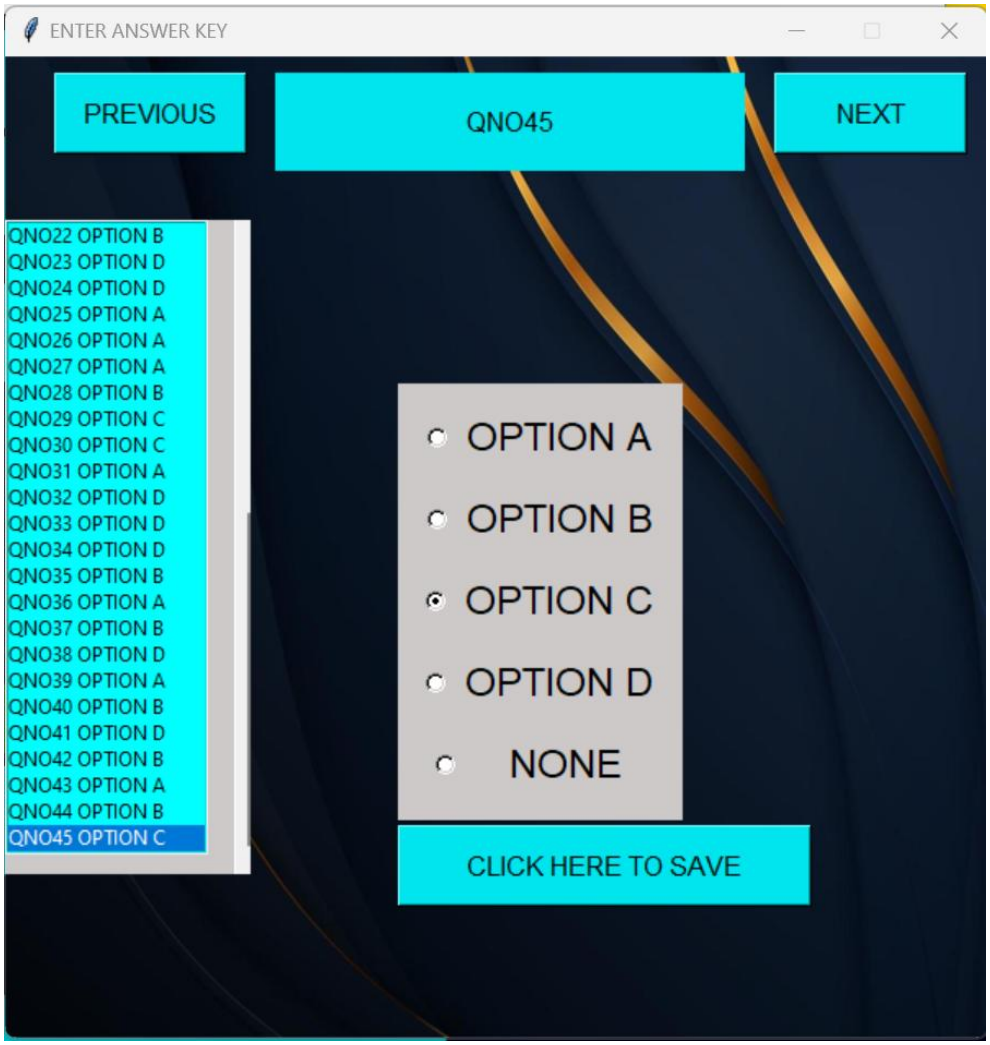
```
NONE  ", activebackground="springgreen",bg="snow3",width=10,height=2,font=("Brittanic
Bold",18))
            N_button.place(x=240, y=400)
            store_anskey_button = Button(enter_anskey_window, text="CLICK HERE TO
SAVE", command=store_anskey, width=27,
                                 height=2,
                                 bg="turquoise2",
                                 font=("Brittanic Bold", 12), )
            store_anskey_button.place(x=240, y=470)

            for i in range(1, no_of_q + 1):
               opt=""
                     try:
                  opt=" OPTION "+anskey[i]
               except:
                  pass


                     lb1.insert(END, "QNO" + str(i)+opt)

            scrollbar1 = ttk.Scrollbar(frame, orient=VERTICAL, command=lb1.yview)
            scrollbar1.place(x=140, y=0, relheight=1)
            lb1.configure(yscrollcommand=scrollbar1.set)

            lb1.bind('<<ListboxSelect>>', select_qno)



         else:
            status_label.configure(bg="red", text="NUMBER OF QUESTIONS MUST BE
POSITIVE INTEGER")

      except:
         status_label.configure(bg="red",text="NUMBER OF QUESTIONS MUST BE
POSITIVE INTEGER")

   def create_test():
      global une
      ans_tablename = str(une.get().replace(" ", "_")) + "ans_key"
         stu_tablename=str(une.get().replace(" ", "_")) + "students"

         db = ms.connect(
         host="localhost",
         user="root",
         password="arunesh7",
         database="CS_PROJECT"
         )
```

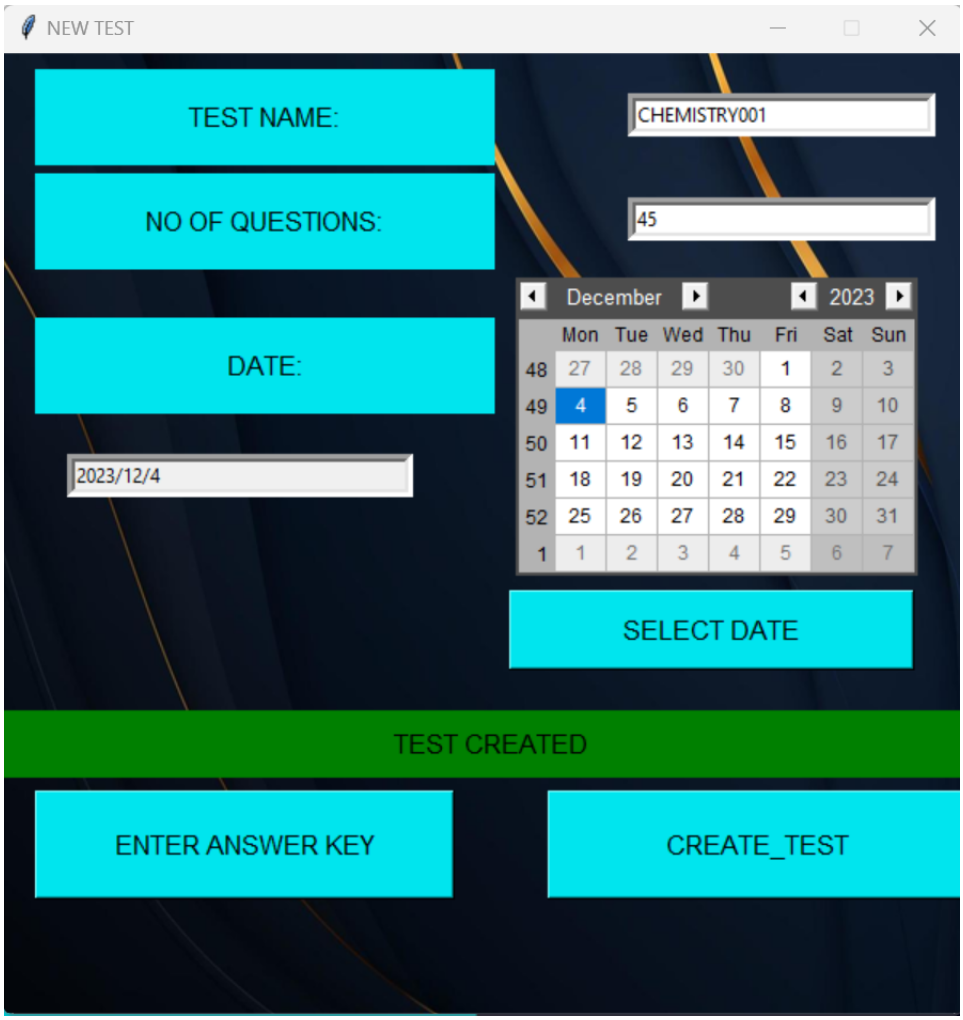
```
        cursor = db.cursor()
        global no_of_q,anskey,anskey_str
        t_n=tne.get()
        date_=date_show_entry.get().replace("/","-")

        if t_n and date_ and no_of_q==len(anskey):

            cursor.execute("SELECT * FROM " + ans_tablename + f" WHERE TEST_NAME =
'{t_n}' ;")
            a = cursor.fetchone()
            if a:status_label.configure(bg="red", text="TEST NAME ALREADY EXISTS")
            else:
                status_label.configure(bg="green", text="TEST CREATED")
                sql=f"insert into {ans_tablename} values('{t_n}','{anskey_str}','{date_}');"
                    cursor.execute(sql)
                db.commit()
                sql2=f"alter table {stu_tablename} add {t_n}_marks integer ,add {t_n}qdetails
VARCHAR(1024);"
                    cursor.execute(sql2)
                db.commit()
                q = None
                    r2 = StringVar()
                r2.set("N")
                anskey = {}
                no_of_q = None
                    anskey_str = None



            else:
            print(no_of_q,t_n,date_,len(anskey))
            status_label.configure(bg="red", text="ENTER ALL DETAILS(TESTNAME,NO OF
QS,ANS KEY,DATE)")


    global new_test_window, img3
    new_test_window = Toplevel(window, )
    new_test_window.title("NEW TEST")
    new_test_window.geometry("600x600")
    new_test_window.resizable(height=False, width=False)
    bgl7= tk.Label(new_test_window, image=img3)
    bgl7.place(x=0, y=0, relwidth=1, relheight=1)
    tnl = tk.Label(new_test_window, text="TEST NAME:", width=27, height=2, pady=10,
padx=20, bg="turquoise2", font=("Brittanic Bold", 12), ).place(x=20, y=10)
    tne = tk.Entry(new_test_window, width=30, borderwidth=5, )
    tne.place(x=390, y=25)
    nql = tk.Label(new_test_window, text="NO OF QUESTIONS:", width=27, height=2,
```

```
pady=10, padx=20, bg="turquoise2",
                font=("Brittanic Bold", 12)).place(x=20, y=75)
    nqe = tk.Entry(new_test_window, width=30, borderwidth=5)
    nqe.place(x=390, y=90)
    status_label = tk.Label(new_test_window, text="ENTER YOUR DETAILS", width=67,
height=2, font=("Brittanic Bold", 12))
    status_label.place(x=0, y=410)
    date_label = tk.Label(new_test_window, text="DATE:", width=27, height=2, pady=10,
padx=20, bg="turquoise2",
                font=("Brittanic Bold", 12))
    date_label.place(x=20, y=165)
    date_show_entry = tk.Entry(new_test_window, width=34,
borderwidth=5,state="readonly",)
    date_show_entry.place(x=40, y=250)
    date=datetime.now()


cal=Calendar(new_test_window,selectmode="day",day=date.day,month=date.month,year=dat
e.year,date_pattern="y/mm/d")
    cal.place(x=320,y=140)
    def choose_date():
        date_show_entry.configure(state="normal")
        date_show_entry.delete(0,END)
        date_show_entry.insert(0,cal.get_date())
        date_show_entry.configure(state="readonly")




    select_date_button=tk.Button(new_test_window, text="SELECT DATE",
command=choose_date,width=27,height=2,bg="turquoise2",font=("Brittanic Bold",12))
    select_date_button.place(x=316,y=335)
    enter_anskey_button = tk.Button(new_test_window, text="ENTER ANSWER KEY",
command=enter_anskey, width=28, height=3, bg="turquoise2",
                        font=("Brittanic Bold", 12))
    enter_anskey_button.place(x=20, y=460)
    create_test_button = tk.Button(new_test_window, text="CREATE_TEST",
command=create_test, width=28, height=3,
                               bg="turquoise2",
                               font=("Brittanic Bold", 12))
    create_test_button.place(x=340, y=460)
```

```
mysql> select TEST_NAME,TEST_DATE from graavitons_11_neetans_key;
+------------+------------+
| TEST_NAME  | TEST_DATE  |
+------------+------------+
| BIOLOGY001 | 2023-11-27 |
| PHYSICS001 | 2023-11-28 |
+------------+------------+
2 rows in set (0.00 sec)
```




ENTER ANSWER KEY
PREVIOUS
QNO45
NEXT
QNO22 OPTION B
QNO23 OPTION D
QNO24 OPTION D
QNO25 OPTION A
QNO26 OPTION A
QNO27 OPTION A
QNO28 OPTION B
QNO29 OPTION C
QNO30 OPTION C
QNO31 OPTION A
QNO32 OPTION D
QNO33 OPTION D
QNO34 OPTION D
QNO35 OPTION B
QNO36 OPTION A
QNO37 OPTION B
QNO38 OPTION D
QNO39 OPTION A
QNO40 OPTION B
QNO41 OPTION D
QNO42 OPTION B
QNO43 OPTION A
QNO44 OPTION B
QNO45 OPTION C
OPTION A
OPTION B
OPTION C
OPTION D
NONE
CLICK HERE TO SAVE




```
mysql> select test_name,test_date from  graavitons_11_neetans_key;
+--------------+------------+
| test_name    | test_date  |
+--------------+------------+
| BIOLOGY001   | 2023-11-27 |
| PHYSICS001   | 2023-11-28 |
| CHEMISTRY001 | 2023-12-04 |
+--------------+------------+
3 rows in set (0.00 sec)
```

8)EDIT TEST MODULE

```
def edit_ans_key():

    def change_anskey():


        global une
        ans_tablename = str(une.get().replace(" ", "_")) + "ans_key"


            db = ms.connect(
            host="localhost",
            user="root",
            password="arunesh7",
            database="CS_PROJECT"
            )
        cursor = db.cursor()
        old_test_name = str(otne.get())

        if old_test_name:
            cursor.execute(f"select * from {ans_tablename}  where TEST_NAME =
'{old_test_name}';")
            a = cursor.fetchone()
            if a:
                try:
                    no_of_q=nqe1.get()
                    if no_of_q != "":
                        no_of_q = int(no_of_q)
                    if no_of_q == "":
                        cursor.execute(f"select ans_key from {ans_tablename} where test_name
= '{old_test_name}'")
                        no_of_q = len(str_to_dict(cursor.fetchone()[0]))

                    if no_of_q > 0:
                        status_label1.configure(bg="green", text="ENTER ANS KEY,SAVE AND
CREATE TEST")

                        edit_anskey_window = Toplevel(main_window)

                        edit_anskey_window.geometry("600x600")
                        edit_anskey_window.title("EDIT ANSWER KEY ")
                        edit_anskey_window.resizable(height=False, width=False)
                        bgl6 = tk.Label(edit_anskey_window, image=img3)
                        bgl6.place(x=0, y=0, relwidth=1, relheight=1)
                        global q
                        q = 1
```

```
                def next():
                    global q

                    if q < no_of_q:
                        q += 1
                        qno_label.configure(text="QNO" + str(q))

                def previous():
                    global q
                    if q > 1:
                        q -= 1
                        qno_label.configure(text="QNO" + str(q))

                def select_qno(e):
                    global q
                    r = lb1.get(ACTIVE).split()[0]
                    q = int(r[3:])

                    qno_label.configure(text="QNO" + str(q))

                qno_label = Label(edit_anskey_window, text="QNO" + str(q), width=27,
height=2, pady=10, padx=20,
                                  bg="turquoise2",
                                  font=("Brittanic Bold", 12), )
                qno_label.place(x=165, y=10)
                next_button = Button(edit_anskey_window, text="NEXT", command=next,
width=12, height=2,
                                     bg="turquoise2",
                                     font=("Brittanic Bold", 12), )
                next_button.place(x=470, y=10)
                previous_button = Button(edit_anskey_window, text="PREVIOUS",
command=previous, width=12,
                                         height=2,
                                         bg="turquoise2",
                                         font=("Brittanic Bold", 12), )
                previous_button.place(x=30, y=10)
                frame = Frame(edit_anskey_window, bg="snow3", width=150, height=400)
                frame.place(x=0, y=100)

                lb1 = Listbox(frame, width=20, height=24, bg="cyan", )
                lb1.place(x=0, y=0)

                def store_anskey():

                    global r2, anskey, anskey_str
                    global une,ANSKEYCHANGED
```

```
                ANSKEYCHANGED = True
                        ans_tablename = str(une.get().replace(" ", "_")) + "ans_key"
                        stu_tablename = str(une.get().replace(" ", "_")) + "students"

                        db = ms.connect(
                    host="localhost",
                    user="root",
                    password="arunesh7",
                    database="CS_PROJECT"
                        )
                cursor = db.cursor()
                old_test_name = str(otne.get())
                if old_test_name:
                    cursor.execute(
                        f"select ANS_KEY from {ans_tablename}  where TEST_NAME =
'{old_test_name}';")
                    a = cursor.fetchone()


                    if a:
                        otans_key = a[0]
                        anskey = str_to_dict(otans_key)
                        optn=r2.get()
                        anskey[q] = optn
                        val = lb1.get(q - 1).split()[0]
                        val += " OPTION " + optn
                        lb1.delete(q - 1)
                        lb1.insert(q - 1, val)
                        anskey_str = dict_to_str(anskey)
                        sql = f"update {ans_tablename} set ANS_KEY = '{anskey_str}'
where TEST_NAME = '{old_test_name}' ;"
                                cursor.execute(sql)
                        db.commit()

                    else:
                        status_label1.configure(bg="red", text="NO SUCH TEST NAME
EXISTS")

                else:
                    status_label1.configure(bg="red", text="ENTER OLD TEST NAME")


            A_button = Radiobutton(edit_anskey_window, variable=r2, value="A",
text="OPTION A",
                                   activebackground="springgreen", bg="snow3", width=10,
height=2,
                                   font=("Brittanic Bold", 18), )
```

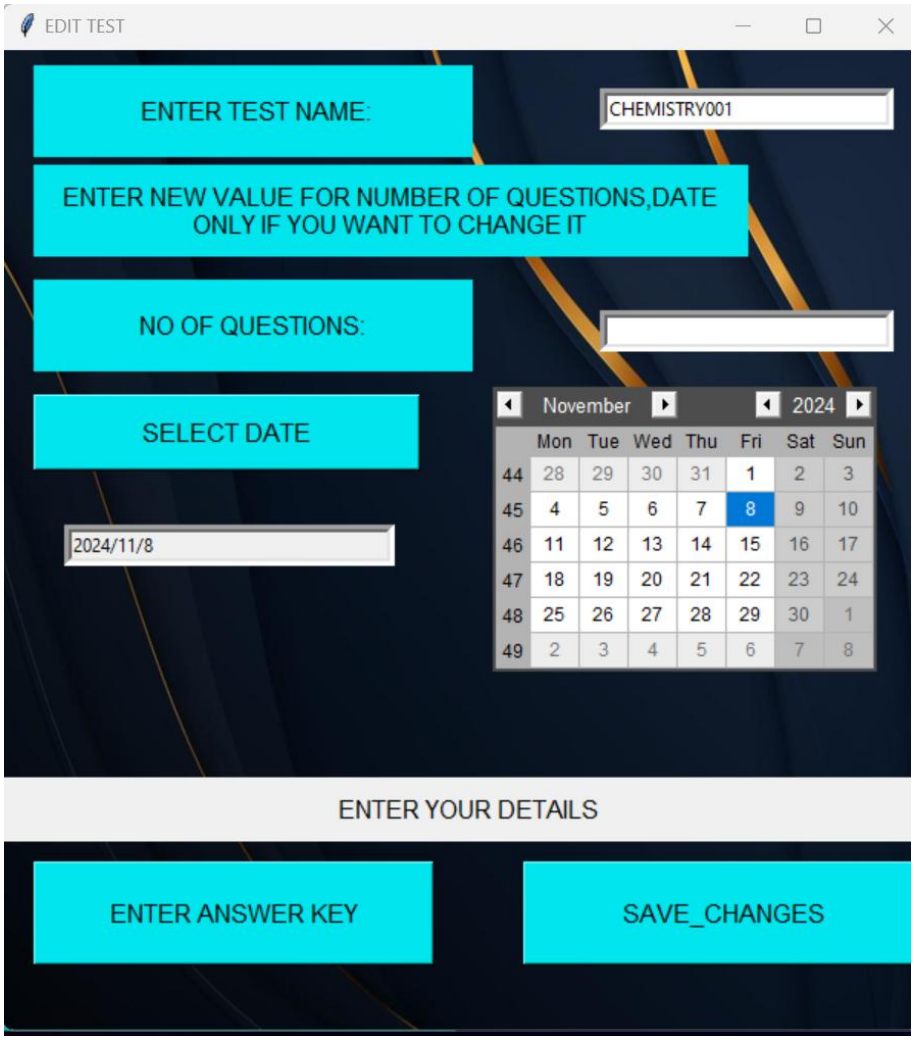
```
                A_button.place(x=240, y=200)
                B_button = Radiobutton(edit_anskey_window, variable=r2, value="B",
text="OPTION B",
                                       activebackground="springgreen", bg="snow3", width=10,
height=2,
                                       font=("Brittanic Bold", 18))
                B_button.place(x=240, y=250)
                C_button = Radiobutton(edit_anskey_window, variable=r2, value="C",
text="OPTION C",
                                       activebackground="springgreen", bg="snow3", width=10,
height=2,
                                       font=("Brittanic Bold", 18))
                C_button.place(x=240, y=300)
                D_button = Radiobutton(edit_anskey_window, variable=r2, value="D",
text="OPTION D",
                                       activebackground="springgreen", bg="snow3", width=10,
height=2,
                                       font=("Brittanic Bold", 18))
                D_button.place(x=240, y=350)
                N_button = Radiobutton(edit_anskey_window, variable=r2, value="N", text="
NONE",
                                       activebackground="springgreen", bg="snow3", width=10,
height=2,
                                       font=("Brittanic Bold", 18))
                N_button.place(x=240, y=400)
                store_anskey_button = Button(edit_anskey_window, text="CLICK TO
SAVE", command=store_anskey, width=27,
                                             height=2,
                                             bg="turquoise2",
                                             font=("Brittanic Bold", 12), )
                store_anskey_button.place(x=240, y=470)
                display2=tk.Label(edit_anskey_window, text="CLICK OPTION AND SAVE
ONLY FOR THE QUESTIONS\nYOU NEED TO CHANGE ANSWER KEY", width=47,
height=2, pady=10, padx=20, bg="turquoise2",
                font=("Brittanic Bold", 12), ).place(x=100, y=520)

                ans_tablename = str(une.get().replace(" ", "_")) + "ans_key"
                        stu_tablename = str(une.get().replace(" ", "_")) + "students"

                        db = ms.connect(
                    host="localhost",
                    user="root",
                    password="arunesh7",
                    database="CS_PROJECT"
                        )
                cursor = db.cursor()
                old_test_name = str(otne.get())
```

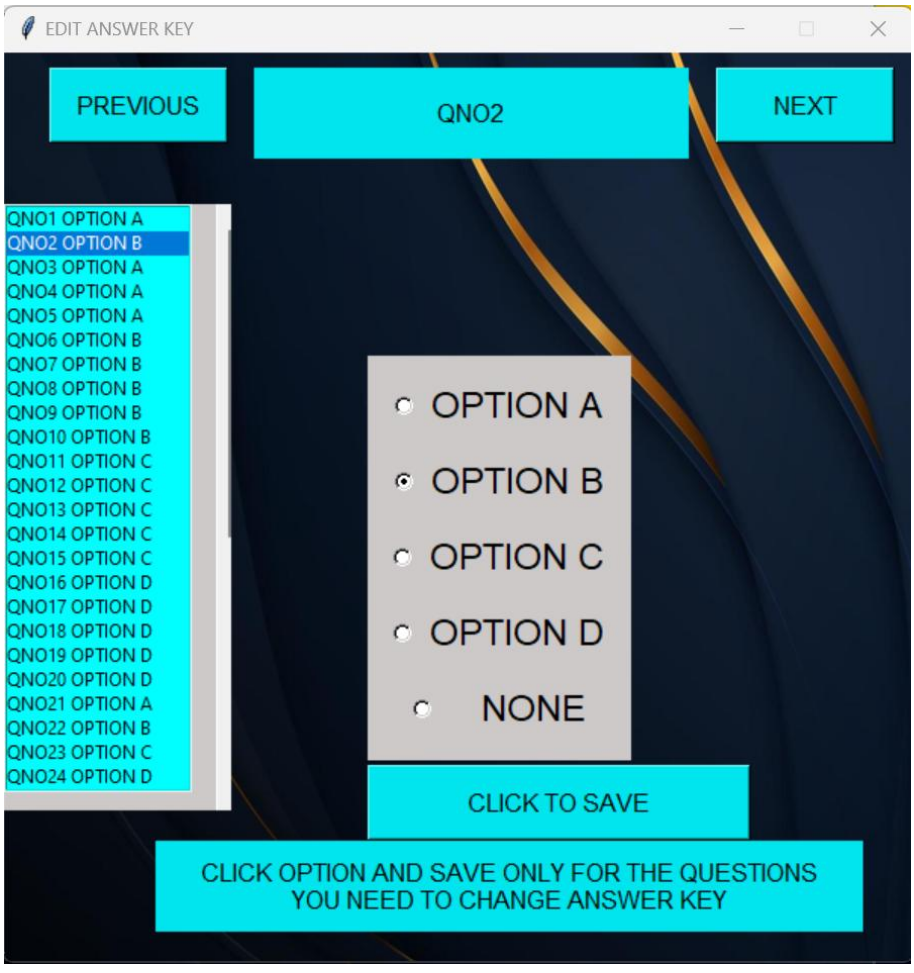
```
                if old_test_name:
                    cursor.execute(
                        f"select ANS_KEY from {ans_tablename}  where TEST_NAME =
'{old_test_name}';")
                    a = cursor.fetchone()

                    if a:
                        otans_key = a[0]
                        anskey = str_to_dict(otans_key)
                for i in range(1, no_of_q + 1):
                    opt = ""
                            try:
                        opt = " OPTION " + anskey[i]
                    except:
                        pass
                            lb1.insert(END, "QNO" + str(i)+opt)

                scrollbar1 = ttk.Scrollbar(frame, orient=VERTICAL, command=lb1.yview)
                scrollbar1.place(x=140, y=0, relheight=1)
                lb1.configure(yscrollcommand=scrollbar1.set)

                lb1.bind('<<ListboxSelect>>', select_qno)


            if no_of_q<0 or no_of_q==0 :
                status_label1.configure(bg="red", text="NUMBER OF QUESTIONS MUST
BE POSITIVE INTEGER")

            except:
                status_label1.configure(bg="red", text="NUMBER OF QUESTIONS MUST
BE POSITIVE INTEGER")

        else:
            status_label1.configure(bg="red", text="NO SUCH TEST EXISTS. CHECK OLD
TEST NAME")
    else:
        status_label1.configure(bg="red", text="OLD TESTNAME MUST BE FILLED
BEFORE ANS KEY")



  def alter_anskey():
    global une,ANSKEYCHANGED
    ans_tablename = str(une.get().replace(" ", "_")) + "ans_key"
        stu_tablename = str(une.get().replace(" ", "_")) + "students"
```

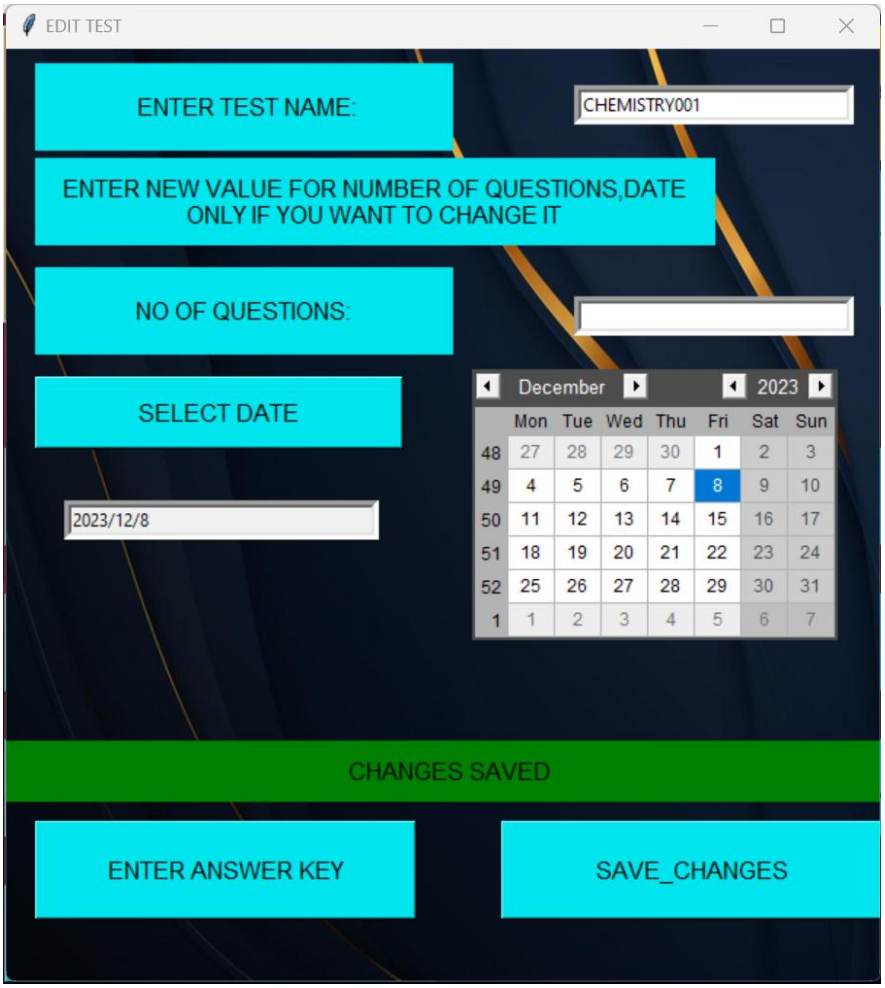
```
        db = ms.connect(
        host="localhost",
        user="root",
        password="arunesh7",
        database="CS_PROJECT"
        )
      cursor = db.cursor()
      global no_of_q, anskey, anskey_str
      try:
        no_of_q = nqe1.get()


        old_test_name = otne.get()
        cursor.execute(f"select TEST_NAME from {ans_tablename}  where TEST_NAME
='{old_test_name}';")
        otn = cursor.fetchone()
        date_ = date_show_entry1.get().replace("/", "-")
        if old_test_name:
          cursor.execute(
            f"select ANS_KEY from {ans_tablename}  where TEST_NAME =
'{old_test_name}';")
          a = cursor.fetchone()

          if a:
            otans_key = a[0]
            anskey = str_to_dict(otans_key)
        if otn:
          if not date_:
            cursor.execute(f"select Test_date from {ans_tablename} where test_name
= '{old_test_name}'")
            date_=cursor.fetchone()[0]
          if no_of_q!="":
            no_of_q=int(no_of_q)
          if no_of_q=="":
            cursor.execute(f"select ans_key from {ans_tablename} where test_name =
'{old_test_name}'")
            no_of_q = len(str_to_dict(cursor.fetchone()[0]))

          if date_ and no_of_q == len(anskey) and old_test_name:
            '''cursor.execute("SELECT TEST_NAME FROM " + ans_tablename + f"
WHERE TEST_NAME =   '{t_n}' ;")
                a = cursor.fetchone()
                if a and a != otn:
                    status_label1.configure(bg="red", text="TEST NAME ALREADY
EXISTS")
                else:'''
                status_label1.configure(bg="green", text="CHANGES SAVED")
```

```
            if ANSKEYCHANGED:
                sql = f"update  {ans_tablename} set
ans_key='{anskey_str}',test_date='{date_}' where test_name='{old_test_name}';"
                if not ANSKEYCHANGED:
                sql = f"update  {ans_tablename} set test_date='{date_}' where
test_name='{old_test_name}';"


                    cursor.execute(sql)
            db.commit()
            """sql2 = f"alter table {stu_tablename} rename COLUMN
{old_test_name}_MARKS to {t_n}_MARKS ; "
                    sql3 = f"alter table {stu_tablename} rename COLUMN
{old_test_name}qdetails to {t_n}qdetails ;"
                    cursor.execute(sql2)
                    db.commit()
                    cursor.execute(sql3)
                    db.commit()"""

              if not (date_ and no_of_q == len(anskey) and old_test_name):
            status_label1.configure(bg="red", text="ENTER ALL
DETAILS(TESTNAME,NO OF QS,ANS KEY,DATE)")

        else:
            status_label1.configure(bg="red", text="OLD TEST NAME DOESNT EXIST")




    except TypeError:

        status_label1.configure(bg="red", text="NUMBER OF QUESTIONS MUST BE
POSITIVE INTEGER")

  global edit_test_window,img3_

  edit_test_window = Toplevel(main_window )
  edit_test_window.title("EDIT TEST")
  edit_test_window.geometry("600x640")
  bgl8 = tk.Label(edit_test_window, image=img3_)
  bgl8.place(x=0, y=0, relwidth=1, relheight=1)

  otnl=tk.Label(edit_test_window, text=" ENTER TEST NAME:", width=27, height=2,
pady=10, padx=20, bg="turquoise2",
            font=("Brittanic Bold", 12), ).place(x=20, y=10)
  otne=tk.Entry(edit_test_window, width=30, borderwidth=5, )
```

```
    otne.place(x=390, y=25)
    displaylabel = tk.Label(edit_test_window, text="ENTER NEW VALUE FOR NUMBER OF
QUESTIONS,DATE\nONLY IF YOU WANT TO CHANGE IT", width=47, height=2,
pady=10, padx=20, bg="turquoise2",
                 font=("Brittanic Bold", 12), ).place(x=20, y=75)


    nql1 = tk.Label(edit_test_window, text="NO OF QUESTIONS:", width=27, height=2,
pady=10, padx=20, bg="turquoise2",
                font=("Brittanic Bold", 12)).place(x=20, y=150)
    nqe1 = tk.Entry(edit_test_window, width=30, borderwidth=5)
    nqe1.place(x=390, y=170)
    status_label1 = tk.Label(edit_test_window, text="ENTER YOUR DETAILS", width=67,
height=2, font=("Brittanic Bold", 12))
    status_label1.place(x=0, y=475)

    date_show_entry1 = tk.Entry(edit_test_window, width=34, borderwidth=5,
state="readonly", )
    date_show_entry1.place(x=40, y=310)
    date = datetime.now()

    cal = Calendar(edit_test_window, selectmode="day", day=date.day, month=date.month,
year=date.year,
                date_pattern="y/mm/d")
    cal.place(x=320, y=220)

    def choose_date():
        date_show_entry1.configure(state="normal")
        date_show_entry1.delete(0, END)
        date_show_entry1.insert(0, cal.get_date())
        date_show_entry1.configure(state="readonly")

    select_date_button1 = tk.Button(edit_test_window, text="SELECT DATE",
command=choose_date, width=27, height=2,
                            bg="turquoise2", font=("Brittanic Bold", 12))
    select_date_button1.place(x=20, y=225)
    enter_anskey_button1 = tk.Button(edit_test_window, text="ENTER ANSWER KEY",
command=change_anskey, width=28, height=3,
                             bg="turquoise2",
                             font=("Brittanic Bold", 12))
    enter_anskey_button1.place(x=20, y=530)
    save_modified_test_button1 = tk.Button(edit_test_window, text="SAVE_CHANGES",
command=alter_anskey, width=28, height=3,
                            bg="turquoise2",
                            font=("Brittanic Bold", 12))
    save_modified_test_button1.place(x=340, y=530)
```


EDIT TEST
ENTER TEST NAME:
CHEMISTRY001
ENTER NEW VALUE FOR NUMBER OF QUESTIONS,DATE ONLY IF YOU WANT TO CHANGE IT
NO OF QUESTIONS:
SELECT DATE
2024/11/8
November
2024
Mon Tue Wed Thu Fri Sat Sun
44 28 29 30 31 1 2 3
45 4 5 6 7 8 9 10
46 11 12 13 14 15 16 17
47 18 19 20 21 22 23 24
48 25 26 27 28 29 30 1
49 2 3 4 5 6 7 8
ENTER YOUR DETAILS
ENTER ANSWER KEY
SAVE_CHANGES


EDIT ANSWER KEY
PREVIOUS
QNO2
NEXT
QNO1 OPTION A
QNO2 OPTION B
QNO3 OPTION A
QNO4 OPTION A
QNO5 OPTION A
QNO6 OPTION B
QNO7 OPTION B
QNO8 OPTION B
QNO9 OPTION B
QNO10 OPTION B
QNO11 OPTION C
QNO12 OPTION C
QNO13 OPTION C
QNO14 OPTION C
QNO15 OPTION C
QNO16 OPTION D
QNO17 OPTION D
QNO18 OPTION D
QNO19 OPTION D
QNO20 OPTION D
QNO21 OPTION A
QNO22 OPTION B
QNO23 OPTION C
QNO24 OPTION D
OPTION A
OPTION B
OPTION C
OPTION D
NONE
CLICK TO SAVE
CLICK OPTION AND SAVE ONLY FOR THE QUESTIONS YOU NEED TO CHANGE ANSWER KEY

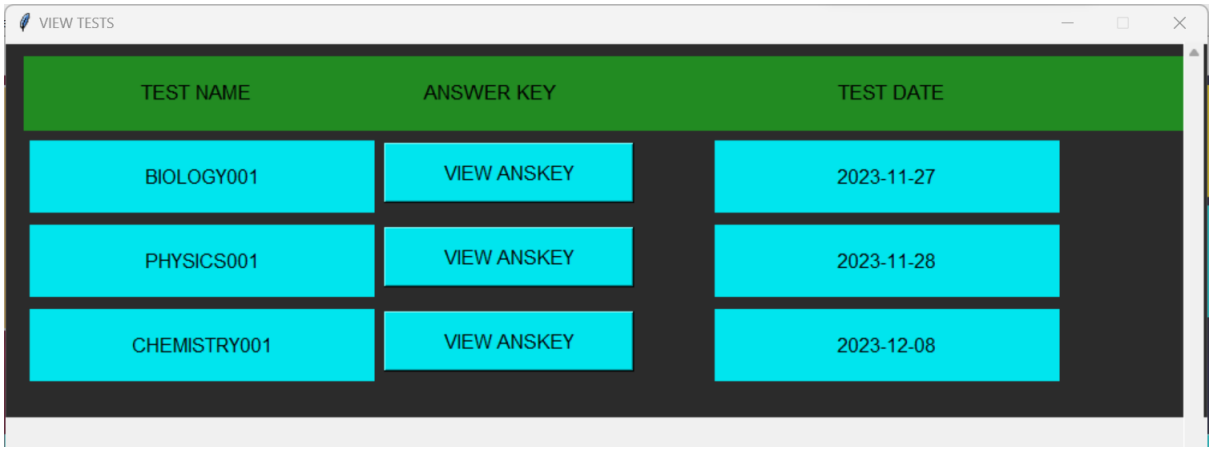


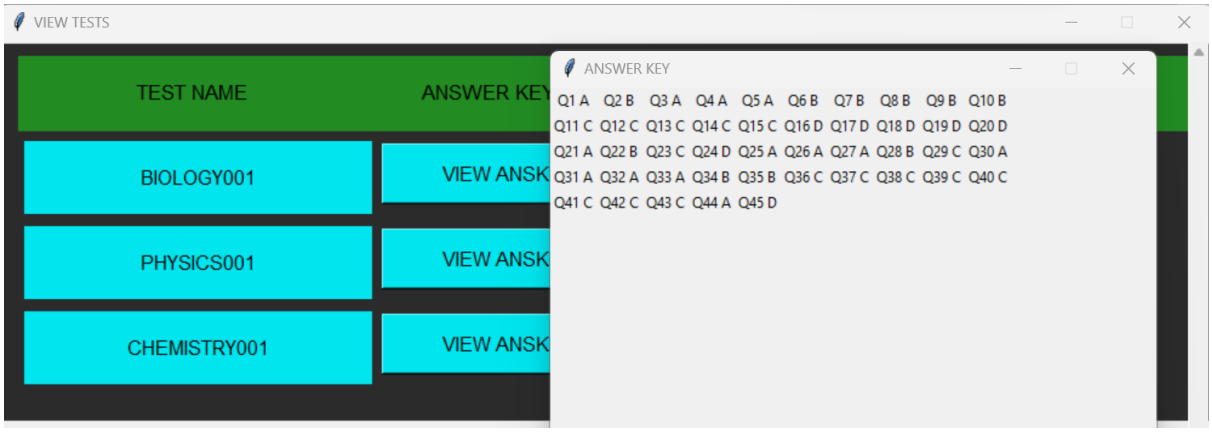

```
mysql> select test_name,test_date from  graavitons_11_neetans_key;
+--------------+------------+
| test_name    | test_date  |
+--------------+------------+
| BIOLOGY001   | 2023-11-27 |
| PHYSICS001   | 2023-11-28 |
| CHEMISTRY001 | 2023-12-08 |
+--------------+------------+
3 rows in set (0.00 sec)
```

9)VIEW TESTS AND ANSWER KEY MODULE

```
def view_tests():
    global une,view_tests_window
    ans_tablename = str(une.get().replace(" ", "_")) + "ans_key"
     stu_tablename = str(une.get().replace(" ", "_")) + "students"
     view_tests_window=Toplevel(main_window)
    view_tests_window.geometry("1000x1000")
    view_tests_window.title("VIEW TESTS")
    view_tests_window.resizable(width=False,height=False)
    canv = Canvas(view_tests_window, height=800, width=800, scrollregion=(0, 0, 1000, 100))
    canv.place(relx=0, rely=0, relheight=1, relwidth=1)
    frame = Frame(canv, width=1000, height=100, bg="grey17")
    frame.place(relheight=1, relwidth=1)
    canv.create_window((0, 0), window=frame, anchor="nw")
    db = ms.connect(
        host="localhost",
        user="root",
        password="arunesh7",
        database="CS_PROJECT"
     )
    cursor = db.cursor()
    sql = f"select TEST_NAME,ANS_KEY,TEST_DATE from {ans_tablename};"
     cursor.execute(sql)
    a = cursor.fetchall()
    # print(a)
    i = 0
    sbar = Scrollbar(view_tests_window, orient=VERTICAL, )
    sbar.place(x=980, y=0, relheight=1)
    testname_label = tk.Label(frame, text="TEST NAME", width=27, height=2, pady=10,
padx=20,bg="forestgreen",font=("Brittanic Bold", 13), ).place(x=15, y=10)
    anskey_label = tk.Label(frame, text="ANSWER KEY", width=27, height=2, pady=10,
padx=20, bg="forestgreen",font=("Brittanic Bold", 13), ).place(x=260, y=10)
    testdate_label = tk.Label(frame, text="TEST DATE     ", width=50, height=2, pady=10,
padx=20,bg="forestgreen",font=("Brittanic Bold", 13), ).place(x=500, y=10)
    buttons={}
    for rec in a:
        ind=a.index(rec)
        testname1_label = tk.Label(frame, text=rec[0], width=27, height=2, pady=10, padx=20,
bg="turquoise2",font=("Brittanic Bold", 12), ).place(x=20, y=80 + (70 * i))
        date_label = tk.Label(frame, text=rec[2], width=27, height=2, pady=10, padx=20,
bg="turquoise2",font=("Brittanic Bold", 12), ).place(x=590, y=80 + (70 * i))
        buttonname=f"BUTTON{ind}"
```

```
        buttons[buttonname]=Button(frame, text="VIEW ANSKEY", command=lambda inde
=ind: viewanskey(a[inde][1]), width=22, height=2, bg="turquoise2", font=("Brittanic Bold",
12)).place(x=315, y=82 + (70 * i))


        canv.configure(scrollregion=(0, 0, 1000, 100 + 70 * i))
        frame.configure(height=170 + 70 * i)
        i += 1
    frame.update_idletasks()
    canv.configure(yscrollcommand=sbar.set)
    sbar.configure(command=canv.yview)
```


VIEW TESTS
TEST NAME
ANSWER KEY
TEST DATE
BIOLOGY001
VIEW ANSKEY
2023-11-27
PHYSICS001
VIEW ANSKEY
2023-11-28
CHEMISTRY001
VIEW ANSKEY
2023-12-08


VIEW TESTS
TEST NAME
ANSWER KEY
BIOLOGY001
VIEW ANSK
PHYSICS001
VIEW ANSK
CHEMISTRY001
VIEW ANSK
ANSWER KEY
Q1 A Q2 B Q3 A Q4 A Q5 A Q6 B Q7 B Q8 B Q9 B Q10 B
Q11 C Q12 C Q13 C Q14 C Q15 C Q16 D Q17 D Q18 D Q19 D Q20 D
Q21 A Q22 B Q23 C Q24 D Q25 A Q26 A Q27 A Q28 B Q29 C Q30 A
Q31 A Q32 A Q33 A Q34 B Q35 B Q36 C Q37 C Q38 C Q39 C Q40 C
Q41 C Q42 C Q43 C Q44 A Q45 D

10)UPLOAD OMRSHEET AND AUTOEVAULUATION MODULE

```
w=720
h=720
clickpoints=[]
def imgsplit(parts,omr_type):
    if omr_type==1:
        no_of_qs=45
        limitingvalue=1700
    elif omr_type==2:
        no_of_qs=90
        limitingvalue = 2700

    elif omr_type==3:
        no_of_qs=180
        limitingvalue = 2700

    elif omr_type==4:
        no_of_qs=30
        limitingvalue = 1700

    qno=[]
    bubbles=[]
    d={}
    q=1
    t=0
    qs = np.vsplit(parts, no_of_qs)

    for r in qs:
        ans=[]
        m_value=[]
        qno.append(r)
        bubble=np.hsplit(r,4)
        n=1
        for b in bubble:
            t+=1
            m=cv2.countNonZero(b)
            bubbles.append(b)
            if m>=limitingvalue:
                ans.append(n)
                m_value.append(m)
            n+=1
        try:
            max_ = max(m_value)
```

```
            i = m_value.index(max_)
            for z in ans:
                if ans.index(z) == i:
                    ans = [z]
        except:
            pass
            d[q]=ans
        q+=1
    return [qno,bubbles,d]
def corrector(dans,dmarked,cm=4,wm=1):
    for k in dans:
        v=dans[k]
        if v =="N":
            dans[k]=0
        elif v=="A":
            dans[k]=1
        elif v=="B":
            dans[k]=2
        elif v=="C":
            dans[k]=3
        elif v=="D":
            dans[k]=4

    l=len(dans)+1
    c=0
    w=0
    u=0
    cans=[]
    wans=[]
    unans=[]
    print(dans)
    for i in range(1,l):
        if len(dmarked[i])==1:

            if    dans[i]==dmarked[i][0]:
                c+=1
                cans.append(i)
            else:
                w+=1
                wans.append(i)
        elif len(dmarked[i])!=1 or dans[i] ==0:
            u+=1
            unans.append(i)
    marks=c*cm-w*wm
```

```
    print(cans,unans,wans)
    d={"CORRECT QUESTIONS":cans,"WRONG QUESTIONS":wans,"UNATTEMPTED
QUESTIONS":unans,"MARKS":marks}


    return d

def crop(omr_type):
    global imgtresh,dans
    w, h = 720, 720
    if omr_type==1:
        points1 = np.array([clickpoints[0], clickpoints[1], clickpoints[2], clickpoints[3]],
dtype="float32")
        points2 = np.array([clickpoints[4], clickpoints[5], clickpoints[6], clickpoints[7]],
dtype="float32")
        points3 = np.array([clickpoints[8], clickpoints[9], clickpoints[10], clickpoints[11]],
dtype="float32")
        pointsf = np.array([[0, 0], [w, 0], [0, h], [w, h]], dtype="float32")
        pers1 = cv2.getPerspectiveTransform(points1, pointsf)
        pers2 = cv2.getPerspectiveTransform(points2, pointsf)
        pers3 = cv2.getPerspectiveTransform(points3, pointsf)

        imgcrop1 = cv2.warpPerspective(imgtresh, pers1, (w, h))
        imgcrop2 = cv2.warpPerspective(imgtresh, pers2, (w, h))
        imgcrop3 = cv2.warpPerspective(imgtresh, pers3, (w, h))
        parts = np.vstack((imgcrop1, imgcrop2, imgcrop3))

        l = imgsplit(parts,omr_type)

        print(l[2])
        d = corrector(dans, l[2])
    elif omr_type==2:
        points1 = np.array([clickpoints[0], clickpoints[1], clickpoints[2], clickpoints[3]],
dtype="float32")
        points2 = np.array([clickpoints[4], clickpoints[5], clickpoints[6], clickpoints[7]],
dtype="float32")
        points3 = np.array([clickpoints[8], clickpoints[9], clickpoints[10], clickpoints[11]],
dtype="float32")
        points4 = np.array([clickpoints[12], clickpoints[13], clickpoints[14], clickpoints[15]],
dtype="float32")
        pointsf1 = np.array([[0, 0], [w, 0], [0, 1104], [w, 1104]], dtype="float32")
        pointsf=np.array([[0, 0], [w, 0], [0, 1056], [w, 1056]], dtype="float32")
        pers1 = cv2.getPerspectiveTransform(points1, pointsf1)
        pers2 = cv2.getPerspectiveTransform(points2, pointsf1)
```

```
        pers3 = cv2.getPerspectiveTransform(points3, pointsf)
        pers4 = cv2.getPerspectiveTransform(points4, pointsf)
        imgcrop1 = cv2.warpPerspective(imgtresh, pers1, (w, 1104))
        imgcrop2 = cv2.warpPerspective(imgtresh, pers2, (w, 1104))
        imgcrop3 = cv2.warpPerspective(imgtresh, pers3, (w, 1056))
        imgcrop4 = cv2.warpPerspective(imgtresh, pers4, (w, 1056))
        parts = np.vstack((imgcrop1, imgcrop2, imgcrop3,imgcrop4))
        cv2.imshow("h",parts)

        l = imgsplit(parts,omr_type)
        print(l[2])
        d = corrector(dans, l[2])
    elif omr_type==3:

        points1 = np.array([clickpoints[0], clickpoints[1], clickpoints[2], clickpoints[3]],
dtype="float32")
        points2 = np.array([clickpoints[4], clickpoints[5], clickpoints[6], clickpoints[7]],
dtype="float32")
        points3 = np.array([clickpoints[8], clickpoints[9], clickpoints[10], clickpoints[11]],
dtype="float32")
        points4 = np.array([clickpoints[12], clickpoints[13], clickpoints[14], clickpoints[15]],
dtype="float32")
        points5 = np.array([clickpoints[16], clickpoints[17], clickpoints[18], clickpoints[19]],
dtype="float32")
        pointsf = np.array([[0, 0], [w, 0], [0, 1728], [w, 1728]], dtype="float32")
        pers1 = cv2.getPerspectiveTransform(points1, pointsf)
        pers2 = cv2.getPerspectiveTransform(points2, pointsf)
        pers3 = cv2.getPerspectiveTransform(points3, pointsf)
        pers4 = cv2.getPerspectiveTransform(points4, pointsf)
        pers5 = cv2.getPerspectiveTransform(points5, pointsf)
        imgcrop1 = cv2.warpPerspective(imgtresh, pers1, (w, 1728))
        imgcrop2 = cv2.warpPerspective(imgtresh, pers2, (w, 1728))
        imgcrop3 = cv2.warpPerspective(imgtresh, pers3, (w, 1728))
        imgcrop4 = cv2.warpPerspective(imgtresh, pers4, (w, 1728))
        imgcrop5 = cv2.warpPerspective(imgtresh, pers5, (w, 1728))
        parts = np.vstack((imgcrop1, imgcrop2, imgcrop3,imgcrop4,imgcrop5))

        l = imgsplit(parts, omr_type)
        print(l[2])
        d = corrector(dans, l[2])
    elif omr_type==4:
        points1 = np.array([clickpoints[0], clickpoints[1], clickpoints[2], clickpoints[3]],
dtype="float32")
        points2 = np.array([clickpoints[4], clickpoints[5], clickpoints[6], clickpoints[7]],
```

```
dtype="float32")
        pointsf = np.array([[0, 0], [w, 0], [0, h], [w, h]], dtype="float32")
        pers1 = cv2.getPerspectiveTransform(points1, pointsf)
        pers2 = cv2.getPerspectiveTransform(points2, pointsf)
        imgcrop1 = cv2.warpPerspective(imgtresh, pers1, (w, h))
        imgcrop2 = cv2.warpPerspective(imgtresh, pers2, (w, h))
        parts = np.vstack((imgcrop1, imgcrop2))
        l = imgsplit(parts, omr_type)
        print(l[2])
        d = corrector(dans, l[2])
    return d,l[2]

def click(event, x, y, flags, param,):

    global clickpoints,omr_type,img_omr

    if event == cv2.EVENT_LBUTTONDOWN:
        clickpoints.append([x, y])
        cv2.circle(img_omr,(x,y),1,color=(0,0,255),thickness=10)
        cv2.imshow("CLICK CORNERS OF OMR",img_omr)

def proceed(stu_id,omr_type):
    global clickpoints,stu_report_window,stu_shadedoptions
    if omr_type == 1:
        if len(clickpoints) == 12:
            d=crop(omr_type)[0]
            stu_shadedoptions=crop(omr_type)[1]
        else:
            raise Exception
    elif omr_type == 2:
        if len(clickpoints) == 16:
            d=crop(omr_type)[0]
            stu_shadedoptions = crop(omr_type)[1]
        else:
            raise Exception
    elif omr_type == 3:
        if len(clickpoints) == 20:
            d=crop(omr_type)[0]
            stu_shadedoptions = crop(omr_type)[1]
        else:
            raise Exception
    stu_report_window=Toplevel(main_window,bg="gray17")
    stu_report_window.title("EVALUATION DETAILS")
    stu_report_window.geometry("900x400")
```

```
    stu_report_window.resizable(height=False)
    print(d)
    mark=d["MARKS"]
    correctqs = str(d["CORRECT QUESTIONS"])[1:len(str(d["CORRECT QUESTIONS"])) -
1]
    if not correctqs:
        correctqs="NONE"
     elif correctqs:
        correctqs = breakmultilines(str(d["CORRECT QUESTIONS"])[1:len(str(d["CORRECT
QUESTIONS"])) - 1])
    wrongqs=str(d["WRONG QUESTIONS"])[1:len(str(d["WRONG QUESTIONS"]))-1]
    if not wrongqs:
        wrongqs = "NONE"
     elif wrongqs:
        wrongqs = breakmultilines(str(d["WRONG QUESTIONS"])[1:len(str(d["WRONG
QUESTIONS"])) - 1])
    unattqs=str(d["UNATTEMPTED QUESTIONS"])[1:len(str(d["UNATTEMPTED
QUESTIONS"]))-1]
    if not unattqs:
        unattqs = "NONE"
     elif unattqs:
        unattqs = breakmultilines(str(d["UNATTEMPTED
QUESTIONS"])[1:len(str(d["UNATTEMPTED QUESTIONS"])) - 1])
    def save_stumarks():
        global d,une,test_name_toupdatemarks,stu_shadedoptions,clickpoints
        clickpoints=[]
        students_table=une.get().upper().replace(" ","_")+"STUDENTS"
            db = ms.connect(
            host="localhost",
            user="root",
            password="arunesh7",
            database="CS_PROJECT"
            )
        cursor = db.cursor()
        sql=f"update {students_table} set
{test_name_toupdatemarks[0]}_MARKS={mark},{test_name_toupdatemarks[0]}qdetails=
'{correctqs}__{wrongqs}__{unattqs}' where stu_id ='{stu_id}';"
            cursor.execute(sql)
        db.commit()
        cv2.destroyAllWindows()

marksl=Label(stu_report_window,width=28,height=2,bg="yellow",text="MARKS",font=("Brit
tanic Bold",12)).grid(row=0,column=0)
    cansl = Label(stu_report_window, width=28, height=2,bg="green", text="ATTEMPTED
```

```
CORRECT ", font=("Brittanic Bold", 12)).grid(row=0,column=1)
    wansl = Label(stu_report_window, width=28, height=2,bg="red", text="ATTEMPTED
WRONG", font=("Brittanic Bold", 12)).grid(row=0,column=2)
    unansl = Label(stu_report_window, width=28, height=2,
bg="orange",text="UNATTEMPTED", font=("Brittanic Bold", 12)).grid(row=0,column=3)
    marksl_ = Label(stu_report_window, width=28, height=8,fg="yellow",bg="gray17",
text=str(d["MARKS"]), font=("Brittanic Bold", 12)).grid(row=1,column=0)
    cansl_ = Label(stu_report_window, width=28, height=8,
fg="green",bg="gray17",text=correctqs, font=("Brittanic Bold", 12)).grid(row=1, column=1)
    wansl_ = Label(stu_report_window, width=28, height=8,fg="red", bg="gray17",
text=wrongqs, font=("Brittanic Bold", 12)).grid(row=1, column=2)
    unansl_ = Label(stu_report_window, width=28, height=8, fg="orange",bg="gray17",
text=unattqs, font=("Brittanic Bold", 12)).grid(row=1, column=3)

savebtn=Button(stu_report_window,text="SAVE",width=165,height=2,bg="turquoise",font=(
"Brittanic Bold", 12),command=save_stumarks).grid(row=2,column=0,columnspan=7)
def upload_img(stu_id,ans_key,omr_dimensions):
    global une, imgtresh, dans, omr_type, img_omr, infolabel, threshhold_value, omr90,
omr30, omr45, omr180,omr_crop_window
    omr_type = int(omr_dimensions[0])
    dans = str_to_dict(ans_key)
    stu_tablename = str(une.get().replace(" ", "_")) + "students"
     fileopen = filedialog.askopenfilename(initialdir="Downloads", title="SELECT IMAGE")
    if omr_type == 1:
        im = omr45
    elif omr_type == 2:
        im = omr90
    elif omr_type == 3:
        im = omr180
    elif omr_type == 4:
        im = omr30
    w = 720
    h = 720
    img_omr = cv2.imread(fileopen)
    img_omr = cv2.resize(img_omr, (w, h))
    cv2.imshow("CLICK CORNERS OF OMR", img_omr)
    omr_crop_window = Toplevel(upload_omr_photo_window, bg="gray")
    omr_crop_window.title("SAMPLE IMAGE AND PROCEDURE")
    omr_crop_window.geometry("600x800")
    omr_crop_window.resizable(height=False, width=False)
    bgl9 = Label(omr_crop_window, image=im).place(x=0, y=50)
    isrunning = True
     threshhold_value = 100
    def close_window():
```

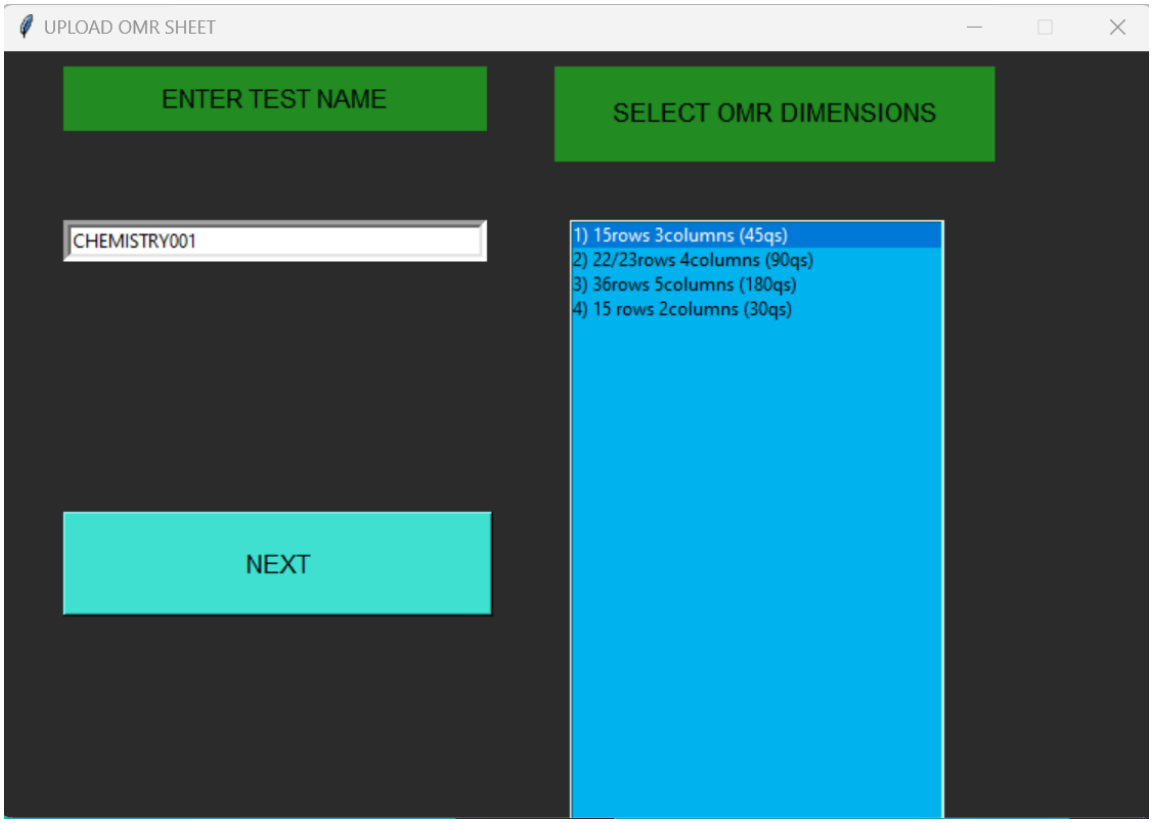
```
        global isrunning
        isrunning = False
            cv2.destroyWindow("CLICK CORNERS OF OMR")
        try:
            proceed(stu_id, omr_type)
        except:
            infolabel.configure(bg="red", text="PLEASE CLICK ALL CORNERS")
    def redo():
        global clickpoints, img_omr
        clickpoints = []
        img_omr = cv2.imread(fileopen)
        img_omr = cv2.resize(img_omr, (w, h))
        cv2.destroyAllWindows()
        cv2.imshow("CLICK CORNERS OF OMR", img_omr)
        cv2.setMouseCallback("CLICK CORNERS OF OMR", click)
    def adjust(event):
        global threshhold_value, imgtresh
        threshhold_value = slider.get()
        imgtresh = cv2.threshold(imggray, threshhold_value, 255,
cv2.THRESH_BINARY_INV)[1]
        cv2.imshow("SET SUITABLE THRESHHOLD VALUE", imgtresh)
    okbtn = Button(omr_crop_window, text="PROCEED", height=2, width=20,
command=close_window,font=("Brittanic Bold", 13), bg="deepskyblue")
    okbtn.place(x=20, y=730)
    redobtn = Button(omr_crop_window, text="REDO", height=2, width=20, command=redo,
font=("Brittanic Bold", 13), bg="deepskyblue")
    redobtn.place(x=200, y=730)
    slider = Scale(omr_crop_window, from_=0, to=255, orient="horizontal", command=adjust,
length=500, width=20)
    slider.place(x=0, y=650)
    infolabel = Label(omr_crop_window, text="SELECT CORNER POINTS OF OMR IN
FOLLOWING ORDER", width=50, height=2,font=("Brittanic Bold", 14), bg="goldenrod")
    infolabel.place(x=0, y=0)
    info2label = Label(omr_crop_window, text="ADJUST THRESHHOLD VALUE\n BEFORE
PROCEEDING", width=30, height=2,font=("Brittanic Bold", 12), bg="deepskyblue")
    info2label.place(x=0, y=600)
    imggray = cv2.cvtColor(img_omr, cv2.COLOR_BGR2GRAY)
    imgtresh = cv2.threshold(imggray, threshhold_value, 255,
cv2.THRESH_BINARY_INV)[1]
    cv2.setMouseCallback("CLICK CORNERS OF OMR", click)
    cv2.waitKey(1)
def upload_omr():
    global une
    ans_tablename = str(une.get().replace(" ", "_")) + "ans_key"
```

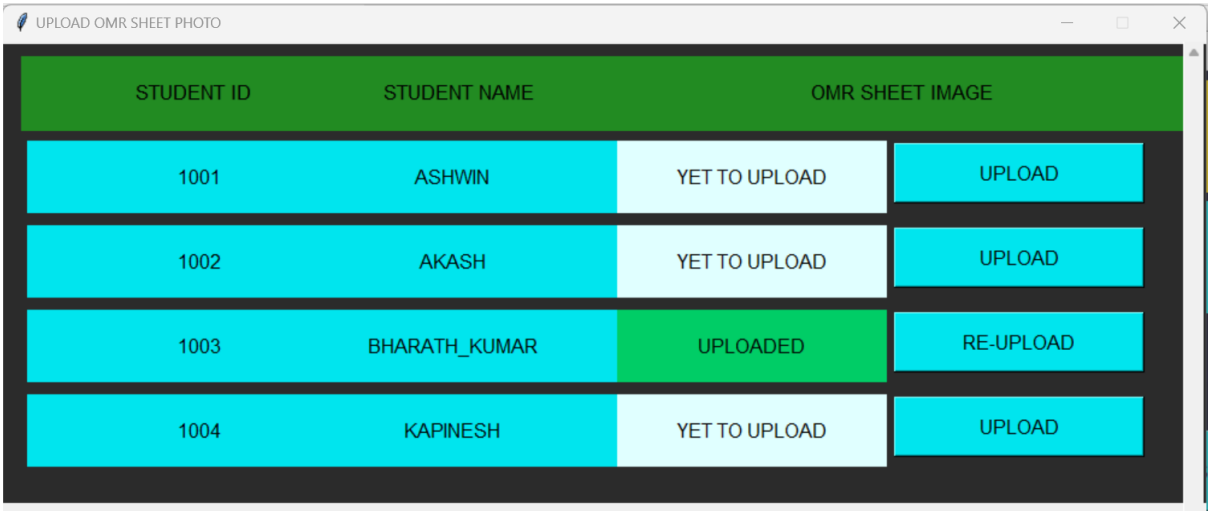
```
    stu_tablename = str(une.get().replace(" ", "_")) + "students"

    global upload_omr_photo_window,ask_test_name_window
   ask_test_name_window=Toplevel(main_window,bg="gray17")
   ask_test_name_window.title("UPLOAD OMR SHEET")
   ask_test_name_window.geometry("750x500")
   ask_test_name_window.resizable(width=False,height=False)
   def enter():
      db = ms.connect(
         host="localhost",
         user="root",
         password="arunesh7",
         database="CS_PROJECT"
         )
      cursor = db.cursor()
      t_n = tn_e.get()
      if t_n:
         cursor.execute(f"select * from {ans_tablename} where TEST_NAME = '{t_n}';
")
         a = cursor.fetchone()
         if a:
            global test_name_toupdatemarks
            test_name_toupdatemarks=a
            ans_key=a[1]
            cursor.execute(f"select * from {stu_tablename}; ")
            a2=cursor.fetchall()


            if a2:
               omr_dimensions=lb.get(ACTIVE)
               upload_omr_photo_window = Toplevel(main_window)
               upload_omr_photo_window.title("UPLOAD OMR SHEET")
               upload_omr_photo_window.geometry("1000x800")
               upload_omr_photo_window.title("UPLOAD OMR SHEET PHOTO")
               upload_omr_photo_window.resizable(height=False, width=False)
               canv = Canvas(upload_omr_photo_window, height=800, width=800,
scrollregion=(0, 0, 1000, 100))
               canv.place(relx=0, rely=0, relheight=1, relwidth=1)
               frame = Frame(canv, width=1000, height=100, bg="grey17")
               frame.place(relheight=1, relwidth=1)
               canv.create_window((0, 0), window=frame, anchor="nw")
               sql = f"select STU_ID,STU_NAME from {stu_tablename};"
                     cursor.execute(sql)
               a = cursor.fetchall()
```

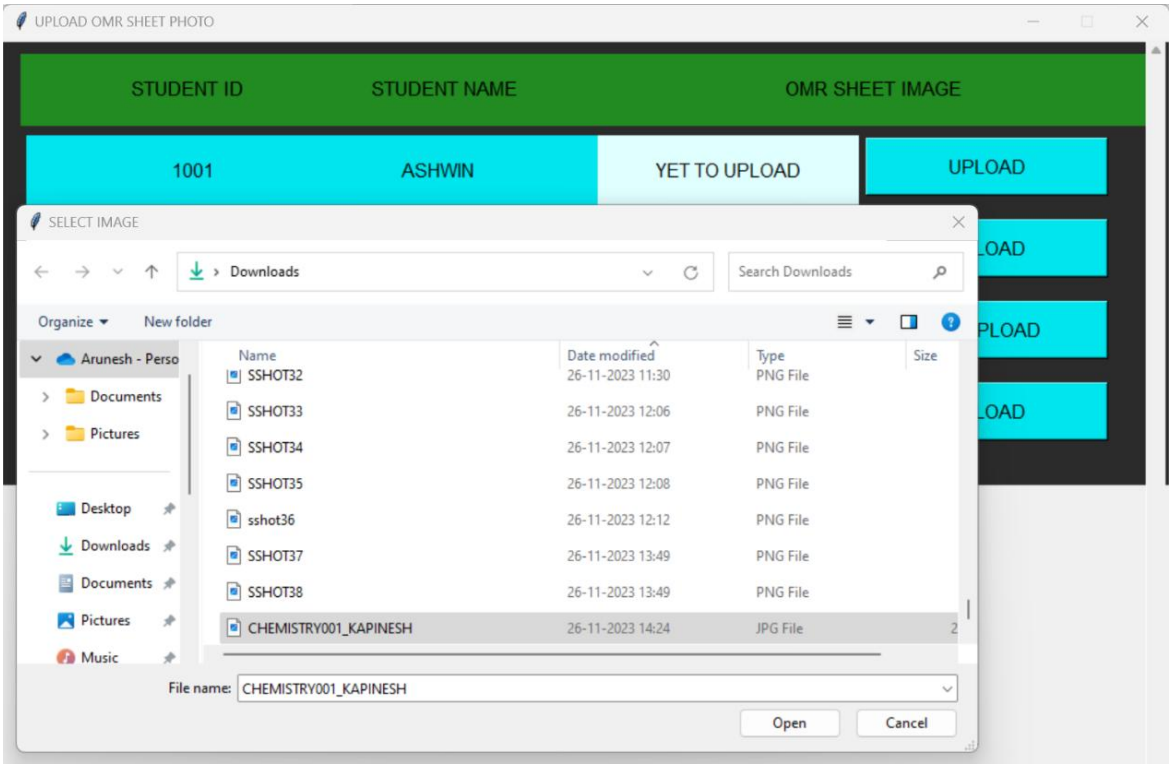
```
            i = 0
            sbar = Scrollbar(upload_omr_photo_window, orient=VERTICAL, )
            sbar.place(x=980, y=0, relheight=1)
            stu_id_label = tk.Label(frame, text="STUDENT ID", width=27, height=2,
pady=10, padx=20,
                                    bg="forestgreen",
                                    font=("Brittanic Bold", 13), ).place(x=15, y=10)
            stu_name_label = tk.Label(frame, text="STUDENT NAME", width=27,
height=2, pady=10, padx=20,
                                      bg="forestgreen",
                                      font=("Brittanic Bold", 13), ).place(x=235, y=10)
            photo_status_label = tk.Label(frame, text="OMR SHEET IMAGE", width=50,
height=2, pady=10, padx=20,
                                        bg="forestgreen",
                                        font=("Brittanic Bold", 13), ).place(x=500, y=10)
            i = 1
            upload_imgbuttons = {}
            for rec in a2:
                ind = a2.index(rec)
                stu_id_label = tk.Label(frame, text=rec[0], width=27, height=2, pady=10,
padx=20,bg="turquoise2",font=("Brittanic Bold", 12), ).place(x=20, y=10 + (70 * i))
                stu_name_label = tk.Label(frame, text=rec[1], width=27, height=2, pady=10,
padx=20,bg="turquoise2", font=("Brittanic Bold", 12), ).place(x=230, y=10 + (70 * i))
                photo_status_label = tk.Label(frame, text="YET TO UPLOAD", width=20,
height=2, pady=10, padx=20, bg="lightcyan", font=("Brittanic Bold", 12), )
                photo_status_label.place(x=510,y=10 + ( 70 * i))
                cursor.execute(f"select {test_name_toupdatemarks[0]}_marks from
{stu_tablename} where STU_ID='{rec[0]}';")
                status=cursor.fetchone()
                button_name = f"BUTTON{ind}"
                        upload_imgbuttons[button_name] = Button(frame, text="UPLOAD",
                                                command=lambda inde=ind:
upload_img(a2[inde][0],ans_key,omr_dimensions,),
                                                width=22,
                                                height=2, bg="turquoise2",
                                                font=("Brittanic Bold", 12))
                upload_imgbuttons[button_name].place(x=740, y=12 + (70 * i))
                if status!=(None,):
                    photo_status_label.configure(text="UPLOADED",bg="spring green3")
                    upload_imgbuttons[button_name].configure(text="RE-UPLOAD")

                canv.configure(scrollregion=(0, 0, 1000, 100 + 70 * i))
                frame.configure(height=100 + 70 * i)
                i += 1
```

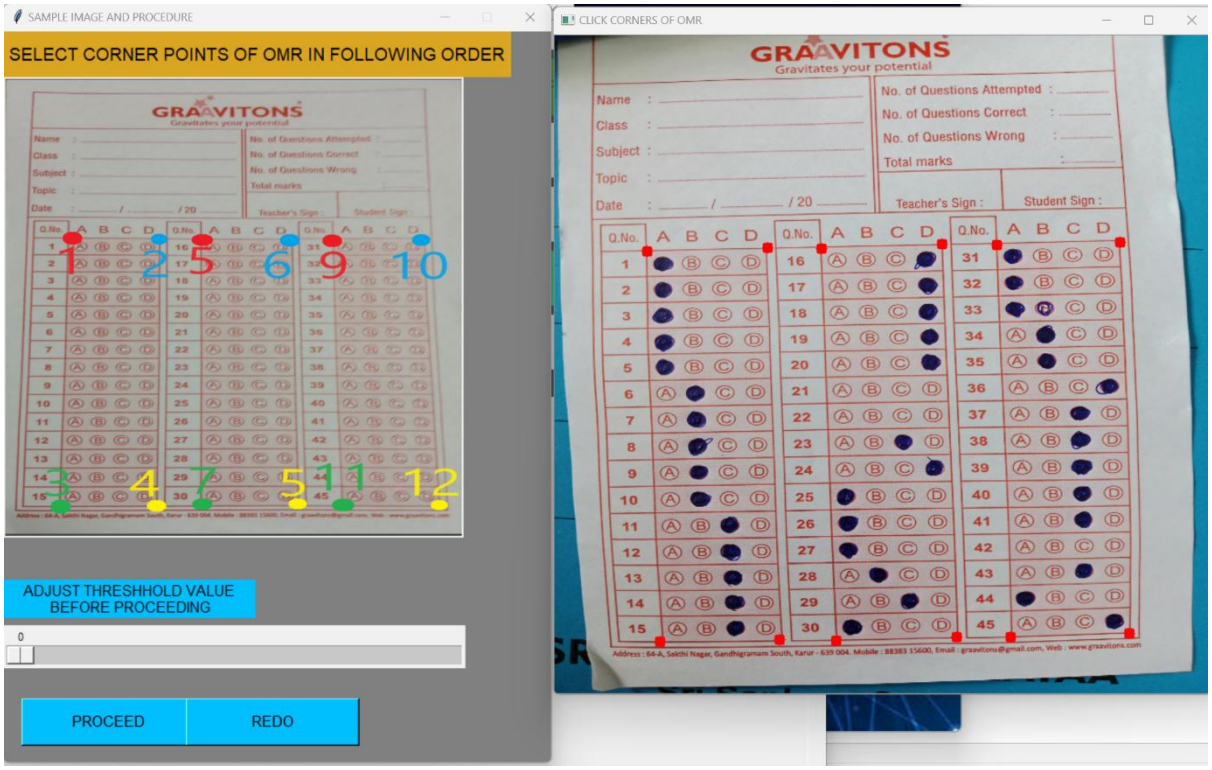
```
                frame.update_idletasks()
                canv.configure(yscrollcommand=sbar.set)
                sbar.configure(command=canv.yview)
            else:
                status_label.configure(text="NO STUDENTS IN DATABASE.ADD THEM
FIRST", bg="red")
        else:
            status_label.configure(text="NO SUCH TEST",bg="red")

    else:
        status_label.configure(text="ENTER TEST NAME",bg="red")


    tn_e = Entry(ask_test_name_window, width=44, borderwidth=5, )
    tn_e.place(x=40, y=110)
    omr_type_label=Label(ask_test_name_window, text="SELECT OMR DIMENSIONS",
width=27, height=2, pady=10, padx=20, bg="forestgreen",font=("Brittanic Bold", 13),
).place(x=360, y=10)
    lb=Listbox(ask_test_name_window,height=70,width=40,bg="deepskyblue2")
    lb.place(x=370,y=110)
    omr_types=["1) 15rows 3columns (45qs)","2) 22/23rows 4columns (90qs)","3) 36rows
5columns (180qs)","4) 15 rows 2columns (30qs)"]
    for typee in omr_types:
        lb.insert(END,typee)
    lb.select_set(0)
    status_label=Label(ask_test_name_window, text="ENTER TEST
NAME",width=30,height=2,bg="forestgreen",font=("Brittanic Bold",12))
    status_label.place(x=40, y=10)

enter_button=Button(ask_test_name_window,width=30,height=3,text="NEXT",command=

enter,font=("Brittanic Bold",12),bg="turquoise")
enter_button.place(x=40,y=300)
```

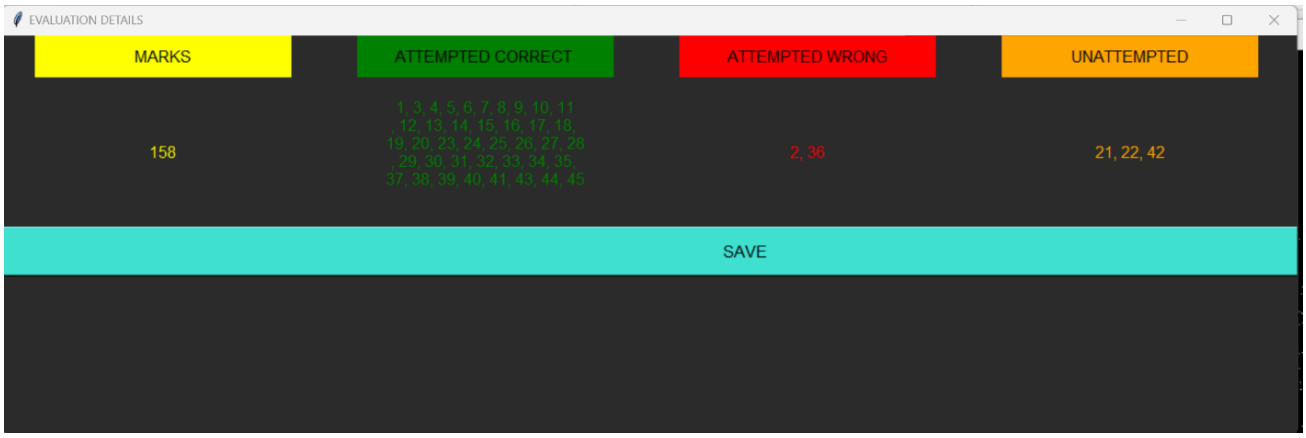
```
mysql> select STU_ID,STU_NAME ,CHEMISTRY001_MARKS FROM  graavitons_11_neetstudents;
+--------+--------------+--------------------+
| STU_ID | STU_NAME     | CHEMISTRY001_MARKS |
+--------+--------------+--------------------+
| 1001   | ASHWIN       |               NULL |
| 1002   | AKASH        |               NULL |
| 1003   | BHARATH_KUMAR |                 30 |
| 1004   | KAPINESH     |               NULL |
+--------+--------------+--------------------+
4 rows in set (0.00 sec)
```






UPLOAD OMR SHEET PHOTO
STUDENT ID
STUDENT NAME
OMR SHEET IMAGE
1001
ASHWIN
YET TO UPLOAD
UPLOAD
SELECT IMAGE
Downloads
Search Downloads
Organize
New folder
Arunesh - Perso
Documents
Pictures
Desktop
Downloads
Documents
Pictures
Music
Name
Date modified
Type
Size
SSHOT32 26-11-2023 11:30 PNG File
SSHOT33 26-11-2023 12:06 PNG File
SSHOT34 26-11-2023 12:07 PNG File
SSHOT35 26-11-2023 12:08 PNG File
sshot36 26-11-2023 12:12 PNG File
SSHOT37 26-11-2023 13:49 PNG File
SSHOT38 26-11-2023 13:49 PNG File
CHEMISTRY001_KAPINESH 26-11-2023 14:24 JPG File
File name: CHEMISTRY001_KAPINESH
Open
Cancel


SAMPLE IMAGE AND PROCEDURE
SELECT CORNER POINTS OF OMR IN FOLLOWING ORDER
ADJUST THRESHHOLD VALUE BEFORE PROCEEDING
PROCEED
REDO
CLICK CORNERS OF OMR
GRAVITONS
Gravitates your potential




```
mysql> select STU_ID,STU_NAME ,CHEMISTRY001_MARKS FROM  graavitons_11_neetstudents;
+--------+--------------+---------------------+
| STU_ID | STU_NAME     | CHEMISTRY001_MARKS |
+--------+--------------+---------------------+
| 1001   | ASHWIN       |                NULL |
| 1002   | AKASH        |                NULL |
| 1003   | BHARATH_KUMAR |                  30 |
| 1004   | KAPINESH     |                 158 |
+--------+--------------+---------------------+
4 rows in set (0.00 sec)
```

11)VIEW STUDENTS MARKS MODULE

```
def get_stu_testandreturntable(students,tests,sort):
    global une
    students_table = une.get().upper().replace(" ","_") + "STUDENTS"
     anskey_table = une.get().upper().replace(" ","_") + "ANS_KEY"
     db = ms.connect(
       host="localhost",
       user="root",
       password="arunesh7",
       database="CS_PROJECT"
     )
    cursor = db.cursor()
    j=0
    def returntable(a,tests):
        stu_tot_marks = []
        if len(tests) == 1:
            test = tests[0].split("_")[0]
            sql0 = f"select ans_key from {anskey_table} where test_name = '{test}';"
           else:
            new=[]
            for i in tests:
                i=i.split("_")[0]
                new.append(f'{i}')
            tests=tuple(new)
            sql0 = f"select ans_key from {anskey_table} where test_name in {tests};"
           cursor.execute(sql0)
        anskeys = cursor.fetchall()
        test_total = []
        for key in anskeys:
            key=key[0]
            noofq = len(str_to_dict(key))
            marks = 4 * noofq
            test_total.append(marks)
        for stu in a:
            tot = 0
            n = 0
            totmarkpercent = 0
            totaltest=0
            markspertest = []
            for test in range(2, len(stu)):
                v = stu[test]
                if v:
                    marks = v
                    markspertest.append(marks)
                    markpercent = (marks / test_total[n]) * 100
                    n += 1
```

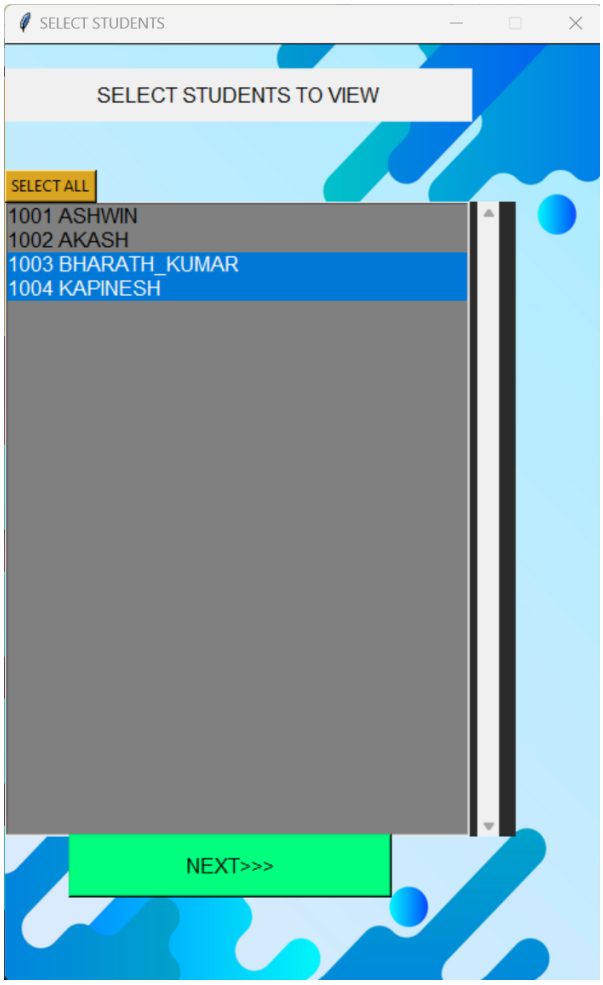
```
                totmarkpercent += markpercent
                totaltest+=1

            else:
                marks_per_test.append(0)
                totaltest+=1
                n+=1
        marks_per_test.append(round((totmarkpercent/totaltest),2))
        stu_tot_marks.append(list(stu[0:2]) + marks_per_test)
    return stu_tot_marks
for i in tests:
    i=i.split()[0]
    tests[j]=i+"_MARKS"
    j+=1
tests=tuple(tests)
stuids=[]
for k in students:
    stuids.append(f'{k[0]}')
stuids=tuple(stuids)
columntuple = ('stu_id', 'stu_name')
columntuple += tests
columntuple=", ".join(columntuple)
if sort=="NAMEASC":
    if len(stuids) == 1:
        stuid = stuids[0]
        sql = f"SELECT {columntuple} from {students_table}  where stu_id = '{stuid}'
order by stu_name ;"
    else:
        sql = f"SELECT {columntuple} from {students_table}  where stu_id in {stuids}
order by stu_name ;"
    cursor.execute(sql)
    a = cursor.fetchall()
    table=returntable(a,tests)
elif sort=="NAMEDESC":
    if len(stuids) == 1:
        stuid = stuids[0]
        sql = f"SELECT {columntuple} from {students_table}  where stu_id = '{stuid}'
order by stu_name DESC ;"
    else:
        sql = f"SELECT {columntuple} from {students_table}  where stu_id in {stuids}
order by stu_name DESC ;"
    cursor.execute(sql)
    a = cursor.fetchall()
    table = returntable(a,tests)
elif sort=="MARKSASC" or sort=="MARKSDESC":
    if len(stuids) == 1:
        stuid = stuids[0]
```

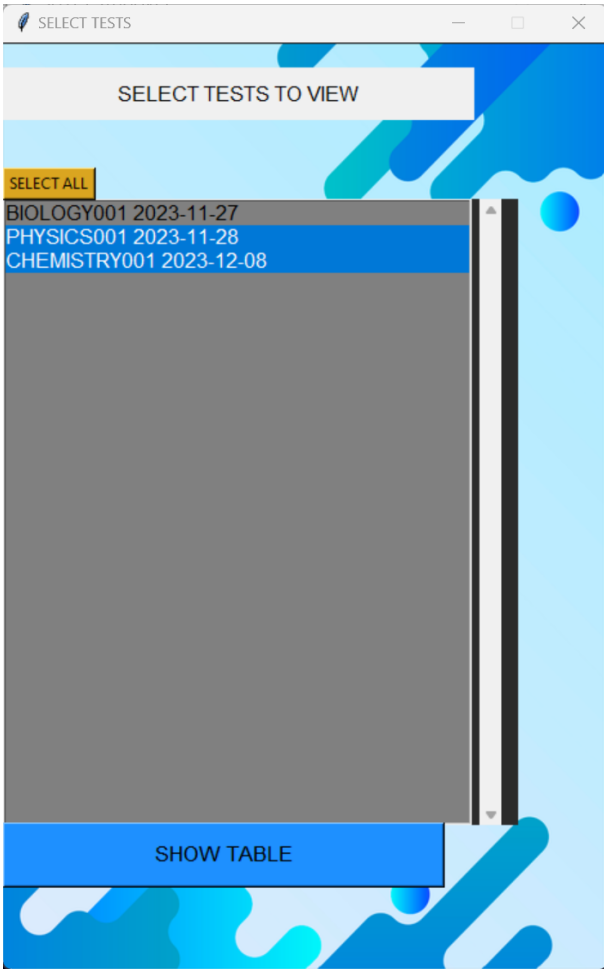
```
            sql = f"SELECT {columntuple} from {students_table}  where stu_id = '{stuid}'  ;"
            else:
            sql = f"SELECT {columntuple} from {students_table}  where stu_id in {stuids} ;"
            cursor.execute(sql)
        a = cursor.fetchall()
        unorderedtable = returntable(a,tests)
        table=[]
        markperc = []
        for rec in unorderedtable:
            markperc.append(rec[-1])
        if sort=="MARKSASC":
            newmarkperc=sorted(markperc)
        else:
            newmarkperc = sorted(markperc,reverse=True)
        for mark in newmarkperc:
            for t in unorderedtable:
                if t[-1]==mark:
                    table.append(t)
    newtable=[]
    for thing in table:
        if thing not in newtable:
            newtable.append(thing)
    return  newtable


def view_students_marks():
    global select_stu_window,view_stu_markwindow,une,img10
    select_stu_window=Toplevel(main_window, )
    students_table = une.get().upper().replace(" ","_") + "STUDENTS"
     anskey_table=une.get().upper().replace(" ","_")+"ANS_KEY"
     db = ms.connect(
       host="localhost",
       user="root",
       password="arunesh7",
       database="CS_PROJECT"
     )
    cursor = db.cursor()
    x=0
    y=0
    select_stu_window.geometry("470x740")
    select_stu_window.title("SELECT STUDENTS")
    select_stu_window.resizable(width=False, height=False)
    bgl10=Label(select_stu_window, image=img10).place(x=0, y=0, relwidth=1, relheight=1)
    information_label=Label(select_stu_window, text="SELECT STUDENTS TO VIEW",
width=40, height=2, font=("Brittanic Bold", 12))
    information_label.place(x=0,y=20)
    stu_frame=Frame(select_stu_window, height=500, width=400, bg="gray17")
```

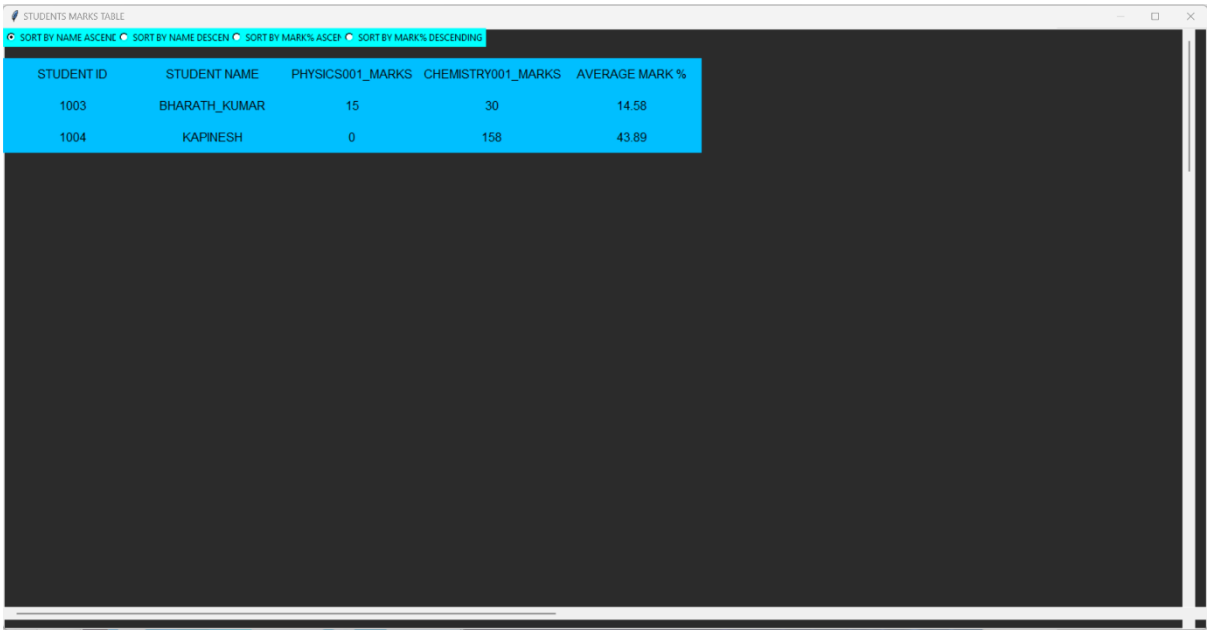
```
    stu_frame.place(x=0,y=125)
    stu_lb=Listbox(stu_frame,height=10,width=40,font=("Brittanic
Bold",12),bg="gray",selectmode="multiple")
    stu_lb.place(x=0,y=0,relheight=1)
    sql1=f"select stu_id,stu_name from {students_table} ; "
     sql2=f"select test_name,test_date from {anskey_table} ;"
     cursor.execute(sql1)
    students=cursor.fetchall()
    cursor.execute(sql2)
    tests=cursor.fetchall()
    for stu in students:
        stu_lb.insert(END,stu)
    sb1=Scrollbar(stu_frame,orient="vertical",command=stu_lb.yview,)
    sb1.place(y=0,x=370,relheight=1)
    def selectallstu():
        for i in range(stu_lb.size()):
            stu_lb.select_set(i)
    selectall1=Button(select_stu_window, bg="goldenrod", text="SELECT ALL",
command=selectallstu, )
    selectall1.place(x=0,y=100)
    stu_lb.configure(yscrollcommand=sb1.set)
    def confirmstu():
        global selected_students,view_tests_window
        b=stu_lb.curselection()#b has indices of all selected items
        selected_students=[stu_lb.get(index) for index in b]
        print(selected_students)
        if selected_students:
            select_test_window = Toplevel(main_window, width=470, height=740)
            select_test_window.title("SELECT TESTS")
            select_test_window.resizable(width=False, height=False)
            bgl10 = Label(select_test_window, image=img10).place(x=0, y=0, relwidth=1,
relheight=1)
            information_label2 = Label(select_test_window, text = "SELECT TESTS TO
VIEW", width = 40, height = 2,font = ("Brittanic Bold", 12))
            information_label2.place(x=0, y=20)
            tests_frame = Frame(select_test_window, width=400, height=500, bg="gray17")
            tests_frame.place(x=0, y=125)
            test_lb = Listbox(tests_frame, height=10, width=40, font=("Brittanic Bold", 12),
bg="gray",
                              selectmode="multiple")
            test_lb.place(x=0, y=0, relheight=1)
            for test in tests:
                print("hello",test,len(test))
                test=test[0]+" "+test[1].strftime('%Y-%m-%d')
                print(test)
                test_lb.insert(END, test)
            sb2 = Scrollbar(tests_frame, orient="vertical", command=test_lb.yview)
```

```
        sb2.place(y=0, x=370, relheight=1)
        sb2 = Scrollbar(tests_frame, orient="vertical", command=test_lb.yview)
        sb2.place(y=0, x=370, relheight=1)
        def selectalltest():
            for i in range(test_lb.size()):
                test_lb.select_set(i)
        selectall2 = Button(select_test_window, bg="goldenrod", text="SELECT ALL",
command=selectalltest, )
        selectall2.place(x=0, y=100)
        test_lb.configure(yscrollcommand=sb2.set)
        def showtable():
            global selected_tests
            a = test_lb.curselection()
            selected_tests = [test_lb.get(index).split()[0] for index in a]
            print("selectedtest",selected_tests)
            if selected_tests:
                global view_stu_markwindow, selected_students
                view_stu_markwindow = Toplevel(main_window,width=1600,height=800 )
                view_stu_markwindow.title("STUDENTS MARKS TABLE")
                canv = Canvas(view_stu_markwindow, height=800, width=1600, scrollregion=(0,
0, 3500, 3500),bg="gray17")
                canv.place(relx=0, rely=0, relheight=1, relwidth=1,x=0,y=0)
                frame = Frame(canv, width=3500, height=3500, bg="grey17")
                frame.place( relheight=1, relwidth=1)
                canv.create_window((0, 40), window=frame,anchor=NW )
                def sortit():
                    view_stu_markwindow.destroy()
                    showtable()
                vbar=Scrollbar(view_stu_markwindow,orient=VERTICAL,)
                vbar.place(x=1570,y=0,relheight=1)
                hbar=Scrollbar(view_stu_markwindow,orient=HORIZONTAL,)
                hbar.place(x=0,y=770,relwidth=1)
                sortasc=Radiobutton(canv,bg="cyan",text="SORT BY NAME
ASCENDING",command=sortit,variable=sort,value="NAMEASC")
                sortasc.place(x=0,y=0)
                sortdesc = Radiobutton(canv, bg="cyan", text="SORT BY NAME
DESCENDING",command=sortit,variable=sort,value="NAMEDESC")
                sortdesc.place(x=150,y=0)
                sortmasc = Radiobutton(canv, bg="cyan", text="SORT BY MARK%
ASCENDING",command=sortit,variable=sort,value="MARKSASC")
                sortmasc.place(x=300,y=0)
                sortmdesc = Radiobutton(canv, bg="cyan", text="SORT BY MARK%
DESCENDING",command=sortit,variable=sort,value="MARKSDESC")
                sortmdesc.place(x=450,y=0)
                table = get_stu_testandreturntable(selected_students,
selected_tests,sort.get())
                stutitle = Label(frame, width=20, height=2, text="STUDENT ID",
```

```
bg="deepskyblue",font=("Brittanic Bold", 12)).grid(row=0,column=0)
            stutitle2 = Label(frame, width=20, height=2, text="STUDENT NAME",
bg="deepskyblue",font=("Brittanic Bold", 12)).grid(row=0,column=1)
            r = 1
            c=2
            for t in selected_tests:
               t_name=Label(frame, width=20, height=2, text=t, bg="deepskyblue",
                        font=("Brittanic Bold", 12)).grid(row=0, column=c)
               c+=1
            avgmarks=Label(frame, width=20, height=2, text="AVERAGE MARK %",
bg="deepskyblue",
                        font=("Brittanic Bold", 12)).grid(row=0, column=c)
            row=1
            for details in table:
               col=0
               for x in details:
                  Label(frame,width=20, height=2, text=str(x), bg="deepskyblue",
                        font=("Brittanic Bold", 12)).grid(row=row,column=col)
                  col+=1
               row+=1
            canv.configure(yscrollcommand=vbar.set, xscrollcommand=hbar.set)
            frame.update_idletasks()
            vbar.configure(command=canv.yview)
            hbar.configure(command=canv.xview)
         else:
            information_label2.configure(bg="red", text="SELECT ATLEAST ONE
TEST")
      showtablebtn = Button(select_test_window, text="SHOW TABLE",
bg="dodgerblue", command=showtable, width=37,
                     height=2, font=("Brittanic Bold", 13))
      showtablebtn.place(x=0, y=623)
   else:
      information_label.configure(bg="red",text="SELECT ATLEAST 1 STUDENT")
  confirmstubtn=Button(select_stu_window, text="NEXT>>>", bg="springgreen",
command=confirmstu, width=27, height=2, font=("Brittanic Bold", 12))
  confirmstubtn.place(x=50,y=623)
```


SELECT STUDENTS
SELECT STUDENTS TO VIEW
SELECT ALL
1001 ASHWIN
1002 AKASH
1003 BHARATH_KUMAR
1004 KAPINESH
NEXT>>>


SELECT TESTS
SELECT TESTS TO VIEW
SELECT ALL
BIOLOGY001 2023-11-27
PHYSICS001 2023-11-28
CHEMISTRY001 2023-12-08
SHOW TABLE


STUDENTS MARKS TABLE
SORT BY NAME ASCENDING
SORT BY NAME DESCENDING
SORT BY MARK% ASCENDING
SORT BY MARK% DESCENDING
STUDENT ID
STUDENT NAME
PHYSICS001_MARKS
CHEMISTRY001_MARKS
AVERAGE MARK %
1003
BHARATH_KUMAR
15
30
14.58
1004
KAPINESH
0
158
43.89

12)ALTER STUDENT MARKS MODULE

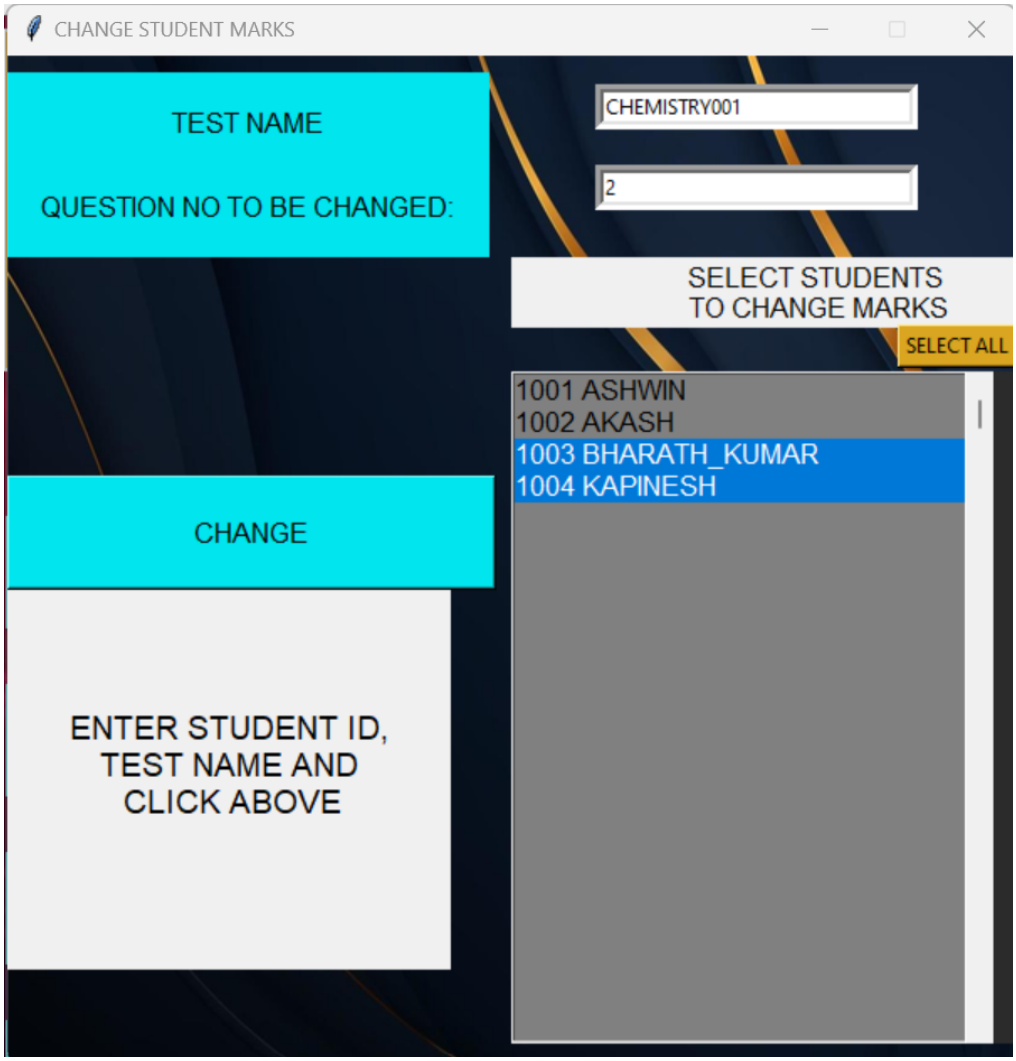
```
def edit_marks():
    global une,edit_marks_window,img3
    students_table = une.get().upper().replace(" ","_") + "STUDENTS"
     anskey_table = une.get().upper().replace(" ","_") + "ANS_KEY"
     db = ms.connect(
       host="localhost",
       user="root",
       password="arunesh7",
       database="CS_PROJECT"
     )
    cursor = db.cursor()
    edit_marks_window=Toplevel(main_window,width=600,height=600)
    edit_marks_window.title("CHANGE STUDENT MARKS")
    edit_marks_window.resizable(width=False,height=False)
    bgl13=Label(edit_marks_window,image=img3).place(relheight=1,relwidth=1)
    qnol = Label(edit_marks_window, text="QUESTION NO TO BE CHANGED:", width=27,
height=2, pady=10, padx=20, bg="turquoise2",
                 font=("Brittanic Bold", 12), ).place(x=0, y=60)
    qnoe = tk.Entry(edit_marks_window, width=30, borderwidth=5, )
    qnoe.place(x=350, y=65)
    tnl=Label(edit_marks_window, text="TEST NAME", width=27, height=2, pady=10,
padx=20, bg="turquoise2", font=("Brittanic Bold", 12), ).place(x=0,y=10)
    t_n_e = tk.Entry(edit_marks_window, width=30, borderwidth=5, )
    t_n_e.place(x=350, y=17)
    infolabel = Label(edit_marks_window, text="ENTER STUDENT ID,\nTEST NAME
AND\n CLICK ABOVE", width=20, height=10,
                      pady=10, padx=20, font=("Brittanic Bold", 14), )
    infolabel.place(x=0, y=300)
    studentslabel = Label(edit_marks_window, text="SELECT STUDENTS \nTO CHANGE
MARKS", width=40, height=2, font=("Brittanic Bold", 12)).place(x=300, y=120)
    stu_frame = Frame(edit_marks_window, height=400, width=300, bg="gray17")
    stu_frame.place(x=300, y=188)
    stu_lb = Listbox(stu_frame, height=10, width=30, font=("Brittanic Bold", 12), bg="gray",
selectmode="multiple")
    stu_lb.place(x=0, y=0, relheight=1)
    sql1 = f"select stu_id,stu_name from {students_table} ; "
     cursor.execute(sql1)
    students = cursor.fetchall()
    for stu in students:
        stu_lb.insert(END, str(stu[0])+" "+stu[1])
    sb1 = Scrollbar(stu_frame, orient="vertical", command=stu_lb.yview, )
    sb1.place(y=0, x=270, relheight=1)
    def selectallstu():
        for i in range(stu_lb.size()):
            stu_lb.select_set(i)
```

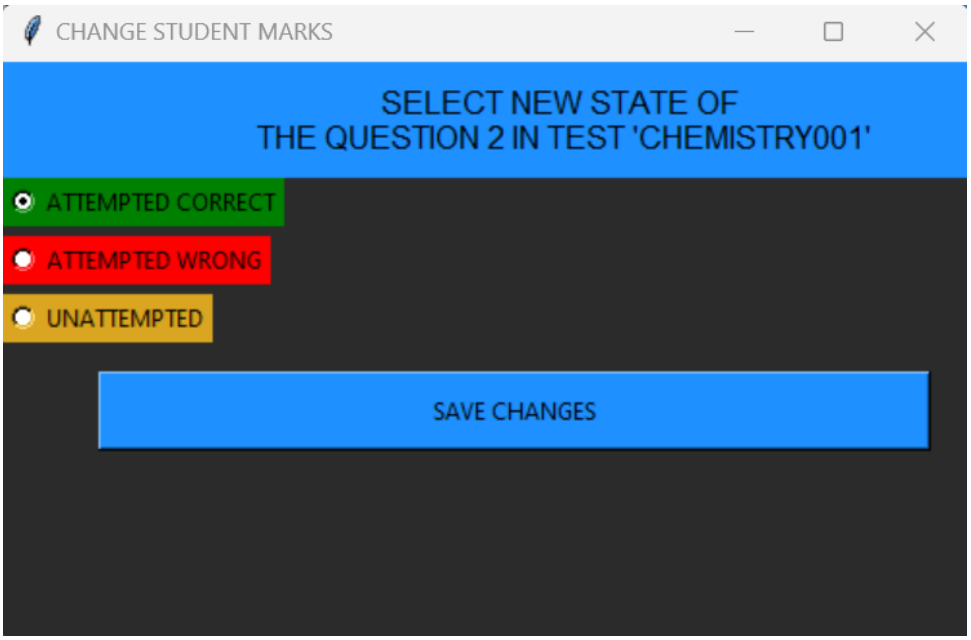
```
   selectall1 = Button(edit_marks_window, bg="goldenrod", text="SELECT ALL",
command=selectallstu )
   selectall1.place(x=530, y=160)
   def change():
      global une, edit_marks_window, img3
      students_table = une.get().upper().replace(" ","_") + "STUDENTS"
         anskey_table = une.get().upper().replace(" ","_") + "ANS_KEY"
         db = ms.connect(
         host="localhost",
         user="root",
         password="arunesh7",
         database="CS_PROJECT"
         )
      cursor = db.cursor()
      b = stu_lb.curselection()
      selected_students = [stu_lb.get(index) for index in b]
      t_n=t_n_e.get().replace(" ","_").upper()
      qno=qnoe.get()
      if t_n and qno and selected_students:
         try:
            if qno.isalnum() and int(qno) > 0:
               cursor.execute(f"select ans_key from {anskey_table} where test_name =
'{t_n}';")
               a = cursor.fetchone()
               if a:
                  nofq = len(str_to_dict(a[0]))
                  if int(qno) > nofq:
                     infolabel.configure(bg="red",
                                 text=f"TEST '{t_n}' HAS \nONLY {str(nofq)}
QUESTIONS\nCHECK ENTERED QNO ")
                  else:
                     ask_new_state_of_q_window = Toplevel(edit_marks_window, width=500,
height=300, bg="gray17")

                     def savechanges():
                        global une, edit_marks_window, img3
                        students_table = une.get().upper().replace(" ","_") + "STUDENTS"
                                 anskey_table = une.get().upper().replace(" ","_") +
"ANS_KEY"
                                 db = ms.connect(
                           host="localhost",
                           user="root",
                           password="arunesh7",
                           database="CS_PROJECT"
                                 )
                        cursor = db.cursor()
                        v = var.get()
```

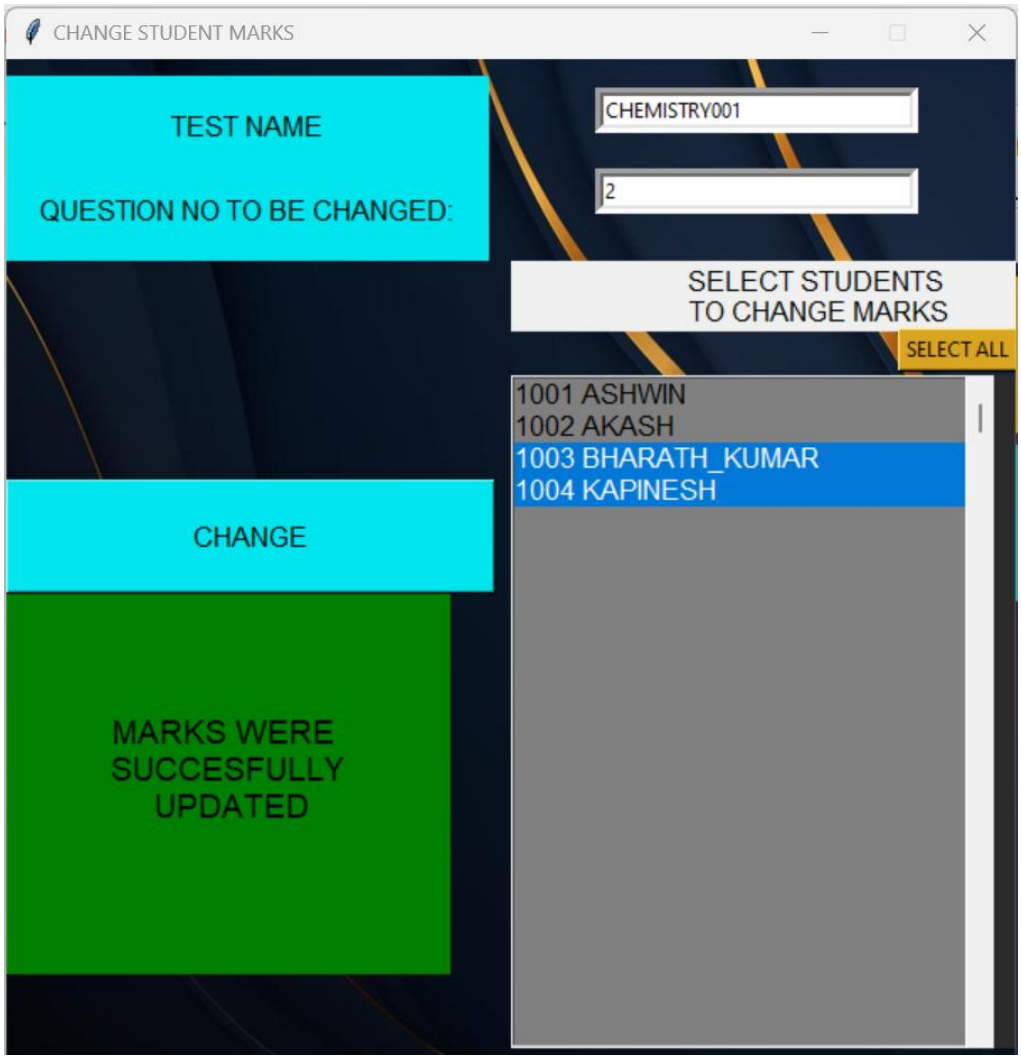
```
                    qno_ = str(qno)
                    for student in selected_students:
                        stuid = student.split()[0]
                        stuname = student.split()[1]
                        cursor.execute(f"select {t_n}qdetails from {students_table} where
stu_id = '{stuid}';")
                        string = cursor.fetchone()[0]
                        if string:
                            c_l = string.split("__")[0].split(",")
                            w_l = string.split("__")[1].split(",")
                            u_l = string.split("__")[2].split(",")
                            if v in ["C", "W", "U"]:
                                if v == "C":
                                    if qno_ not in c_l:
                                        c_l.append(qno_)
                                        if qno_ in w_l:
                                            w_l.remove(qno_)
                                        elif qno_ in u_l:
                                            u_l.remove(qno_)
                                elif v == "W":
                                    if qno_ not in w_l:
                                        w_l.append(qno_)
                                        if qno_ in c_l:
                                            c_l.remove(qno_)
                                        elif qno_ in u_l:
                                            u_l.remove(qno_)
                                elif v == "U":
                                    if qno_ not in u_l:
                                        u_l.append(qno_)
                                        if qno_ in w_l:
                                            w_l.remove(qno_)
                                        elif qno_ in c_l:
                                            c_l.remove(qno_)
                                if "NONE" in c_l:
                                    c_l.remove("NONE")
                                if "NONE" in w_l:
                                    w_l.remove("NONE")
                                if "NONE" in u_l:
                                    u_l.remove("NONE")
                                m = str(4 * len(c_l) - len(w_l))
                                if c_l == []:
                                    c_l = ["NONE,"]
                                if w_l == []:
                                    w_l = ["NONE,"]
                                if u_l == []:
                                    u_l = ["NONE,"]
                                c_s = ",".join(c_l)
```

```
                                w_s = ",".join(w_l)
                                u_s = ",".join(u_l)
                                s = "__".join([c_s, w_s, u_s])
                                cursor.execute(
                                  f"UPDATE {students_table} SET {t_n}_MARKS =
'{m}',{t_n}qdetails = '{s}' where stu_id = '{stuid}';")
                                db.commit()
                                ask_new_state_of_q_window.destroy()
                                infolabel.configure(bg="green", text="MARKS WERE \n
SUCCESFULLY \n UPDATED")
                            else:
                                ques.configure(bg="red")
                        else:
                            ask_new_state_of_q_window.destroy()
                            infolabel.configure(bg="red",
                                                text=f"OMR_PHOTO IS \n NOT UPLOADED
FOR\n'{stuname}'")
                    ques = Label(ask_new_state_of_q_window,text=f"SELECT NEW STATE
OF \nTHE QUESTION {str(qno)} IN TEST '{t_n}'",width=64,height=3, font=("Brittanic
Bold", 12) ,bg="dodgerblue")
                    ques.place(x=0, y=0)
                    crt = Radiobutton(ask_new_state_of_q_window, bg="green",
text="ATTEMPTED CORRECT",variable=var,value="C")
                    wrg = Radiobutton(ask_new_state_of_q_window, bg="red",
text="ATTEMPTED WRONG", variable=var,value="W")
                    unt = Radiobutton(ask_new_state_of_q_window, bg="goldenrod",
text="UNATTEMPTED", variable=var,value="U")
                    crt.place(x=0, y=60)
                    wrg.place(x=0, y=90)
                    unt.place(x=0, y=120)
                    savebtn = Button(ask_new_state_of_q_window, bg="dodgerblue",
text="SAVE CHANGES", width=60,height=2,command=savechanges)
                    savebtn.place(x=50, y=160)
                else:
                    infolabel.configure(bg="red", text="NO SUCH\nTEST NAME ")
            else:
                raise Exception
        except:
            infolabel.configure(bg="red", text="QUESTION NO \nSHOULD BE \n
POSITIVE INTEGER")
    else:
        infolabel.configure(bg="red",text="FILL ALL\n DETAILS")

changebtn=Button(edit_marks_window,text="CHANGE",width=27,height=2,pady=10,padx=2
0,bg="turquoise2",font=("Brittanic Bold",12),command=change)
  changebtn.place(x=0,y=250)
```

```
mysql> select STU_ID,STU_NAME ,CHEMISTRY001_MARKS FROM  graavitons_11_neetstudents;
+--------+--------------+--------------------+
| STU_ID | STU_NAME     | CHEMISTRY001_MARKS |
+--------+--------------+--------------------+
| 1001   | ASHWIN       |               NULL |
| 1002   | AKASH        |               NULL |
| 1003   | BHARATH_KUMAR |                 30 |
| 1004   | KAPINESH     |                158 |
+--------+--------------+--------------------+
4 rows in set (0.00 sec)
```








```
mysql> select STU_ID,STU_NAME ,CHEMISTRY001_MARKS FROM  graavitons_11_neetstudents;
+--------+--------------+--------------------+
| STU_ID | STU_NAME     | CHEMISTRY001_MARKS |
+--------+--------------+--------------------+
| 1001   | ASHWIN       |               NULL |
| 1002   | AKASH        |               NULL |
| 1003   | BHARATH_KUMAR |                 34 |
| 1004   | KAPINESH     |                163 |
+--------+--------------+--------------------+
4 rows in set (0.00 sec)
```

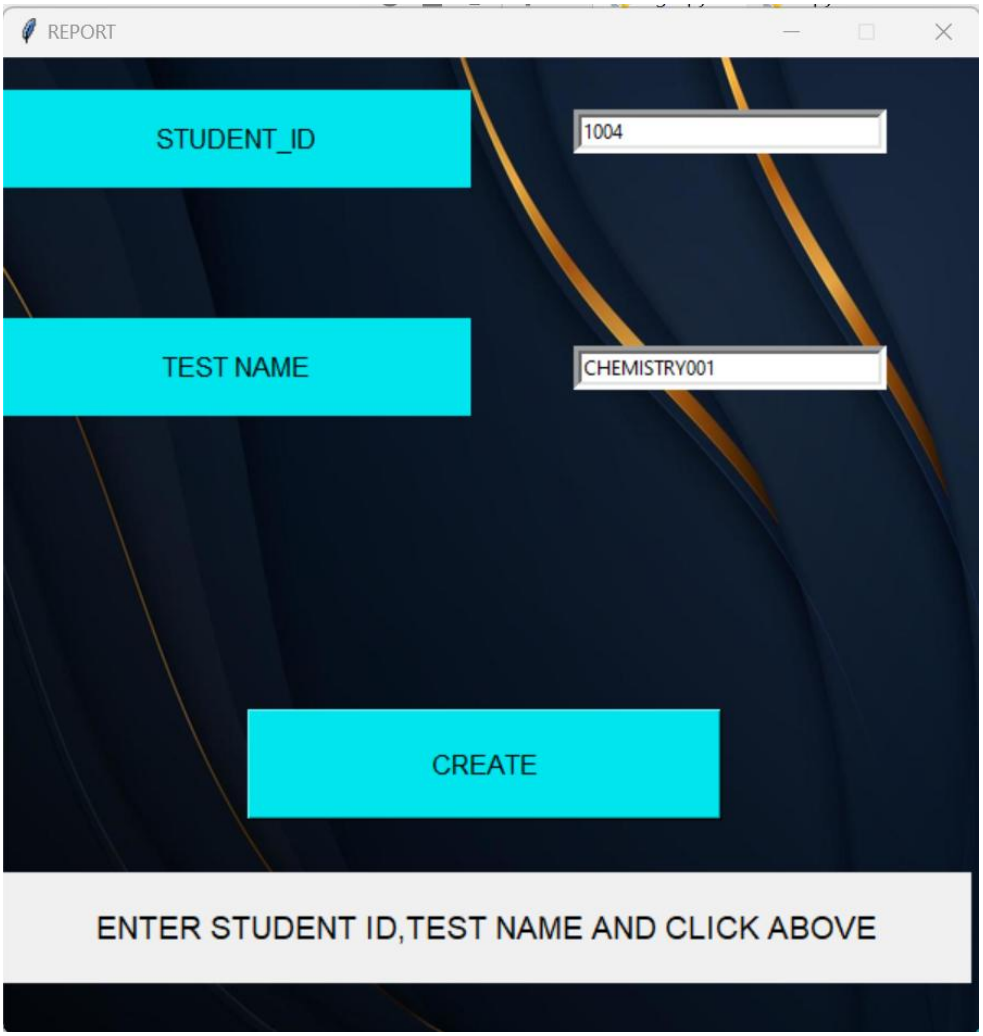
13) CREATE REPORT MODULE

```
def generate_report():
    global img3,une
    def generate():
        global  une
        db = ms.connect(
            host="localhost",
            user="root",
            password="arunesh7",
            database="CS_PROJECT"
            )
        cursor = db.cursor()
        stu_table=une.get().replace(" ","_")+"STUDENTS"
            ans_table=une.get().replace(" ","_")+"ANS_KEY"
            stu=sid_e.get().replace(" ","_")
        t_n=t_n_e.get().replace(" ","_")
        if stu and t_n:
            try:
                cursor.execute(f"select stu_name,{t_n}_MARKS,{t_n}qdetails from {stu_table}
where stu_id ='{stu}';")
                a = cursor.fetchone()
                cursor.execute(f"select test_date,ans_key from {ans_table} where test_name
= '{t_n}';")
                d = cursor.fetchone()
                if a and d:
                    tot_m = str(len(str_to_dict(d[1])) * 4)
                    crct = breakmultilines(a[2].split("__")[0].replace("\n", ""), 110)
                    wrng = breakmultilines(a[2].split("__")[1].replace("\n", ""),110)
                    unat = breakmultilines(a[2].split("__")[2].replace("\n", ""), 110)
                    nofc = str(len(crct.split(",")))
                    nofw = str(len(wrng.split(",")))
                    nofu = str(len(unat.split(",")))
                    if "NONE" in crct:
                        nofc = 0
                    if "NONE" in wrng:
                        nofw = 0
                    if "NONE" in unat:
                        nofu = 0
                    generate_report_2window = Toplevel(main_window, width=900, height=600,
bg="turquoise2")
                    generate_report_2window.title("REPORT")
                    generate_report_2window.resizable(height=False, width=False)
                    idl = Label(generate_report_2window, text="STUDENT ID :",
                                bg="turquoise2", font=("Brittanic Bold", 12), width=30, padx=40,
pady=40).grid(row=0,
                                                                        column=0)
```

```
            Label(generate_report_2window, text=stu.upper(), bg="turquoise2",
font=("Brittanic Bold", 12),
                  width=30, padx=40, pady=40).grid(row=0, column=1, )

            nl = Label(generate_report_2window, text="STUDENT NAME : ",
bg="turquoise2",
                       font=("Brittanic Bold", 12), width=30, padx=40, pady=40).grid(row=1,
column=0)

            Label(generate_report_2window, text=a[0].upper(), bg="turquoise2",
font=("Brittanic Bold", 12),
                  width=30, padx=40, pady=40).grid(row=1, column=1)

            tnl = Label(generate_report_2window, text="TEST NAME : ",
bg="turquoise2",
                        font=("Brittanic Bold", 12), width=30, padx=40, pady=40).grid(row=2,
column=0)

            Label(generate_report_2window, text=t_n.upper() + " " + str(d[0]),
bg="turquoise2",
                  font=("Brittanic Bold", 12), width=30, padx=40, pady=40).grid(row=2,
column=1, )

            marks = Label(generate_report_2window, text=f"MARKS (OUT OF {tot_m}) :
", bg="turquoise2",
                          font=("Brittanic Bold", 12), width=30, padx=40, pady=40).grid(row=3,
column=0)

            Label(generate_report_2window, text=str(a[1]), bg="turquoise2",
font=("Brittanic Bold", 12),
                  width=30, padx=40, pady=40).grid(row=3, column=1, )

            Label(generate_report_2window, text="NO OF QS ", bg="turquoise2",
font=("Brittanic Bold", 12),
                  width=30, padx=40, pady=40).grid(row=4, column=1)

            Label(generate_report_2window, text="Q NUMBERS ", bg="turquoise2",
font=("Brittanic Bold", 12),
                  width=30, padx=40, pady=40).grid(row=4, column=2)

            attcrct = Label(generate_report_2window, text="ATTEMPTED CORRECT : ",
bg="turquoise2",
                            font=("Brittanic Bold", 12), width=30, padx=40,
pady=40).grid(row=5, column=0)

            Label(generate_report_2window, text=nofc, bg="turquoise2", font=("Brittanic
Bold", 12), width=8,
```

```
                padx=40, pady=40).grid(row=5, column=1)

            Label(generate_report_2window, text=crct, bg="turquoise2", font=("Brittanic
Bold", 12), width=70,
                padx=40, pady=40).grid(row=5, column=2)

            attwrng = Label(generate_report_2window, text="ATTEMPTED WRONG : ",
bg="turquoise2",
                        font=("Brittanic Bold", 12), width=30, padx=40,
pady=40).grid(row=6, column=0)

            Label(generate_report_2window, text=nofw, width=8, padx=40, pady=40,
bg="turquoise2",
                font=("Brittanic Bold", 12), ).grid(row=6, column=1)

            Label(generate_report_2window, text=wrng, bg="turquoise2", font=("Brittanic
Bold", 12), width=70,
                padx=40, pady=40).grid(row=6, column=2)

            unatt = Label(generate_report_2window, text="UNATTEMPTED : ",
bg="turquoise2",
                      font=("Brittanic Bold", 12), width=30, padx=40, pady=40).grid(row=7,
column=0)

            Label(generate_report_2window, text=nofu, bg="turquoise2", font=("Brittanic
Bold", 12), width=8,
                padx=40, pady=40).grid(row=7, column=1)

            Label(generate_report_2window, text=unat, bg="turquoise2", font=("Brittanic
Bold", 12), width=70,
                padx=40, pady=40).grid(row=7, column=2)
        else:
            x = ""
                if not a:
                    x += "STUDENT ID,"
                if not d:
                    x += "TEST NAME"

                infolabel.configure(bg="red", text="NO SUCH "+x)

    except:
        infolabel.configure(bg="red", text="RECHECK STUDENT ID ,TEST NAME
MAKE SURE TO\n UPLOAD OMR SHEET PHOTO AND THEN GENERATE REPORT")

else:
    infolabel.configure(bg="red",text="FILL ALL DETAILS")
```

```
    generate_report_1window=Toplevel(main_window,width=600,height=600)
    generate_report_1window.title("REPORT")
    generate_report_1window.resizable(height=False,width=False)
    bgl12=Label(generate_report_1window,image=img3).place(relheight=1,relwidth=1)

stu_id=Label(generate_report_1window,text="STUDENT_ID",width=27,height=2,pady=10,p
adx=20,bg="turquoise2",font=("Brittanic Bold",12),).place(x=0,y=20)
    Test_n = Label(generate_report_1window, text="TEST NAME", width=27, height=2,
pady=10, padx=20, bg="turquoise2",
                font=("Brittanic Bold", 12), ).place(x=0,y=160)
    sid_e = tk.Entry(generate_report_1window, width=30, borderwidth=5, )
    sid_e.place(x=350,y=32)
    t_n_e = tk.Entry(generate_report_1window, width=30, borderwidth=5, )
    t_n_e.place(x=350, y=177)
    infolabel=Label(generate_report_1window,text="ENTER STUDENT ID,TEST NAME
AND CLICK ABOVE",width=50,height=2,pady=10,padx=20,font=("Brittanic Bold",14),)
    infolabel.place(x=0,y=500)

createbtn=Button(generate_report_1window,text="CREATE",width=27,height=2,pady=10,pa
dx=20,bg="turquoise2",font=("Brittanic Bold",12),command=generate)
    createbtn.place(x=150,y=400)
```



REPORT

STUDENT ID :	1004	
STUDENT NAME :	KAPINESH	
TEST NAME :	CHEMISTRY001 2023-12-08	
MARKS (OUT OF 180) :	163	
	NO OF QS	Q NUMBERS
ATTEMPTED CORRECT :	41	1, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33 , 34, 35, 37, 38, 39, 40, 41, 43, 44, 45,2
ATTEMPTED WRONG :	1	36
UNATTEMPTED :	3	21, 22, 42

CONCLUSION

- **Core Deliverable Realization:** Successfully architected and implemented a functional end-to-end OMR processing prototype utilizing **Python, OpenCV, and Tkinter**, effectively bridging automated computer vision matrix analysis with a multi-tenant relational data-logging system.
- **Key Technical Proficiencies Acquired:** Developed deep practical competencies in digital image processing, including spatial coordinate transformation and contour isolation, alongside mastering desktop GUI state management and relational database schema serialization for long-term historical telemetry tracking.
- **Encountered Constraints & Future Scope:** Identified critical accuracy limitations during testing, as variations in ambient lighting heavily distorted the pixel-intensity readings. Because test grading requires absolute precision, the project was paused at the prototype stage, highlighting the clear need for adaptive thresholding to handle real-world lighting in any future version.